マルチクラウド自動デプロイプラットフォーム

日本語解説書

PLAYER1-r7

2026年2月27日

Contents

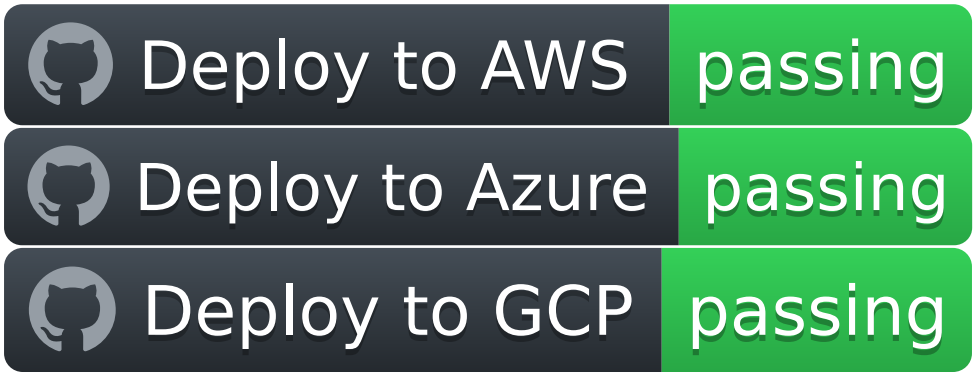
1　プロジェクト概要　2
2　ソース実装解説書　14
3　インフラ・CI/CD解説書　26
4　カスタムドメイン設定　34
5　ステージング環境テスト　43
6　統合テストガイド　50
7　Lambda Layer最適化　58
8　アーキテクチャマッパー　62

Chapter 1

プロジェクト概要

Multi-Cloud

Figure 1.1: Multi-Cloud



マルチクラウド対応の自動デプロイシステム - AWS/Azure/GCP対応のフルスタックアプリケーション自動デ

Cloud Provider	API Endpoint	Frontend (CDN)	Direct Storage
AWS (ap-northeast-1)	API Gateway	CloudFront ☒	S3 Static
Azure (japaneast)	Functions	Front Door ☒	Blob Storage
GCP (asia-northeast1)	Cloud Functions	Cloud CDN ☒	Cloud Storage

Cloud Provider	Resource	セキュリティ実装	状態
AWS	CloudFront + Lambda	CORS 絞り込み + CloudTrail	☒ 本番反映
Azure	Key Vault + Functions	Purge Protection + Managed Identity + 診断ログ	☒ 本番反映
GCP	Cloud Functions	HTTPS redirect + Audit Logs + Cloud Armor	☒ 本番反映

☒ **全クラウドでCDN配信を実装！** CloudFront, Front Door, Cloud CDNによる高速・安全なコンテンツ配信

☒ **Infrastructure as Code**: Pulumiで全CDNリソースを管理（詳細: CDNセットアップガイド）

☒ 詳細情報: エンドポイント一覧

☒ **2026-02-27 更新**: S1 セキュリティ全本番反映完了 / S2 Managed Identity 有効化 / Task 20/21 Key Vault & Defender 実装済み

☒ **2026-02-22 Refactoring**: Infra & CI/CD Refactoring Report — Azure AFD SPA fix, workflow cleanup, Pulumi dead code removal, staging validation

- **マルチクラウド対応**: AWS、Azure、GCPに対応
- **フルスタック**: フロントエンド、バックエンド、データベースの完全なスタック
- **自動デプロイ**: GitHub Actionsによる完全自動化
- **IaC統合**: Pulumi 3.0+による Infrastructure as Code
- **完全Python版**: Pulumi + FastAPI + Reflexによる統一スタック
- **CI/CD**: プッシュやPRで自動的にビルド・デプロイ
- **簡単セットアップ**: スクリプト一つで環境構築

```
multicloud-auto-deploy/ ├──
 .github/workflows/     # GitHub ワークフローActions ├──
 infrastructure/        # インフラストラクチャコード │
    └── pulumi/           # コード（PulumiPython - AWS/Azure/）GCP ├──
 services/             # アプリケーションコード │
    ├── api/              # FastAPI バックエンド（Python ）3.12 │
    ├── frontend_react/   # React フロントエンド（Vite + ）TypeScript │
    └── frontend_reflex/  # Reflex フロントエンド（実験的） ├──
 scripts/              # デプロイ・テストスクリプト ├──
 docs/                 # ドキュメント │
    ├── CDN_SETUP.md                      # 設定ガイドCDN │
    ├── ENDPOINTS.md                      # エンドポイント一覧 │
    └── REFACTORING_REPORT_20260222.md    # 🆕 Infra & CI/CD refactoring report
     (2026-02-22) └──
 static-site/          # 静的サイト（環境選択画面）
```

- Python 3.12+
- Docker & Docker Compose
- Pulumi 3.0+
- AWS CLI 2.x / Azure CLI 2.x / gcloud CLI 556.0+
- GitHub アカウント

Frontend

- **Framework**: React 18+ (Vite)
- **Hosting**: Static Site (S3 / Azure Blob / Cloud Storage)
- **CDN**: CloudFront / Azure Front Door / Cloud CDN

- **Build**: Vite 7.3+, TypeScript

Backend

- **Framework**: FastAPI 1.0+ (Python 3.12)
- **AWS**: Lambda (x86_64) + API Gateway v2 (HTTP)
- **Azure**: Azure Functions (Python)
- **GCP**: Cloud Functions (Python 3.11)

Database

- **AWS**: DynamoDB (PAY_PER_REQUEST)
- **Azure**: Cosmos DB (Serverless)
- **GCP**: Firestore (Native Mode)

Infrastructure

- **IaC**: Pulumi 3.0+
  - Infrastructure as Code管理（`infrastructure/pulumi/`）
    - AWS: Lambda + API Gateway
    - Azure: Functions + Cosmos DB
    - GCP: Cloud Functions
- **CI/CD**: GitHub Actions

CI/CD

- GitHub Actions
- Automated builds and deployments
- Docker-based deployments
- S3-based Lambda deployment

1. **リポジトリをクローン**

```
git clone https://github.com/PLAYER1-r7/multicloud-auto-deploy.git
cd multicloud-auto-deploy
```

2. **ローカル開発環境起動**

```
docker-compose up -d api

cd services/frontend_react
npm install
npm run dev
```

3. **Pulumiでデプロイ**

```
cd infrastructure/pulumi/aws/simple-sns
pip install -r requirements.txt
pulumi stack init staging
pulumi config set aws:region ap-northeast-1
pulumi up
```

☒ 詳細な移行ガイドは docs/PYTHON_MIGRATION.md を参照

1. **リポジトリをクローン**

```
git clone https://github.com/PLAYER1-r7/multicloud-auto-deploy.git
cd multicloud-auto-deploy
```

2. **GitHub Secretsを設定**

```
./scripts/setup-github-secrets.sh
```

3. **コードをプッシュして自動デプロイ**

```
git push origin main
```

- ☒ セットアップガイド - 初期セットアップ手順
- ☒ CI/CD設定 - GitHub Actions自動デプロイ設定
- ☒ CI/CDテスト結果 - パイプライン検証レポート
- ☒ トラブルシューティング - よくある問題と解決策
- ☒ エンドポイント一覧 - 全環境のエンドポイント情報
- ☒ CDNセットアップガイド - CloudFront/Front Door/Cloud CDN設定
- ☒ 関数サイズ最適化 - デプロイパッケージサイズ削減（AWS 97%削減！）
- ☒ クイックリファレンス - よく使うコマンド集
- ☒ セキュリティ実装 - Key Vault / Managed Identity / WAF / CloudTrail / Audit Logs
- AWS デプロイ
- Azure デプロイ
- GCP デプロイ
- システムアーキテクチャ - 完全版システム設計
- クラウドアーキテクチャ可視化 - 公式アイコン付きインタラクティブHTML図生成
  - ☒ Staging環境 - AWS/Azure/GCP公式アイコン統合版
  - ☒ Production環境 - 本番環境構成図
  - ☒ 統合ビュー - Staging + Production比較図

プッシュやPRで自動的にビルド・デプロイが実行されます：

- `main`ブランチへのプッシュ → 全3クラウド本番デプロイ（AWS/Azure/GCP）
- `develop`ブランチへのプッシュ → ステージング環境へデプロイ
- PRの作成/更新 → ビルド検証
- 手動トリガー → 任意の環境へデプロイ

2026-02-27 デプロイ状況:

- ☒ AWS production: 9/0 SNS E2E test PASS
- ☒ Azure production: 17/0 SNS E2E test PASS
- ☒ GCP production: 13/0 SNS E2E test PASS
- ☒ 全環境セキュリティ実装完了（Pulumi反映）

ワークフロー	トリガー	デプロイ先	説明
deploy-aws.yml	`main`へのpush / 手動	AWS Lambda	Lambda + API Gateway更新
deploy-azure.yml	`main`へのpush / 手動	Azure Functions	Functions + Cosmos DB更新
deploy-gcp.yml	`main`へのpush / 手動	GCP Cloud Functions	Cloud Functions更新

以下のシークレットを設定してください（詳細は CI/CD設定ガイド 参照）：

Pulumi（必須）

- `PULUMI_ACCESS_TOKEN` - Pulumi Cloud認証トークン（すべてのデプロイで必須）

Azure Functions

- `AZURE_CREDENTIALS` - Service Principal認証情報（JSON）
- `AZURE_SUBSCRIPTION_ID` - Azure Subscription ID
- `AZURE_RESOURCE_GROUP` - リソースグループ名

GCP Cloud Functions

- `GCP_CREDENTIALS` - サービスアカウントキー（JSON）
- `GCP_PROJECT_ID` - プロジェクトID

AWS Lambda

- `AWS_ACCESS_KEY_ID` - AWS認証キーID
- `AWS_SECRET_ACCESS_KEY` - AWS認証シークレットキー

最新のデプロイ状況はGitHub Actionsで確認でき、詳細タスク進捗も公開中です。

最新ステータス（2026-02-27）： | タイプ | 対象 | 状態 | |——|——|——| | インフラ整備 | AWS/Azure/GCP Pulumi本番 | ☒ デプロイ完了 | | セキュリティ実装 | CORS / CloudTrail / Key Vault / Managed Identity / Audit Logs | ☒ 本番反映完了 | | E2E テスト | 全クラウド

SNS 統合テスト | ☒ 39/39 PASS | | 認証 | AWS Cognito / Azure AD / Firebase | ☒ 全完了 | | CDN | CloudFront / Front Door / Cloud CDN | ☒ 全完了 |

GitHub Actionsページから手動でワークフローを実行：

```
Actions > Deploy to Multi-Cloud > Run workflow

- environment: staging / production
- deploy_target: all / azure / gcp
```

- **Frontend**: S3 Static Website Hosting
- **Backend**: Lambda (Python 3.12) + API Gateway v2
- **Database**: DynamoDB (PAY_PER_REQUEST)
- **Infrastructure**: Pulumi 3.0+
- **Deployment**: GitHub Actions
- **Frontend**: Blob Storage ($web) + Azure Front Door
- **Backend**: Azure Functions (Python 3.12)
- **Database**: Cosmos DB (Serverless)
- **Infrastructure**: Pulumi 3.0+
- **Deployment**: GitHub Actions
- **CDN**: Azure Front Door (Standard)
- **Frontend**: Cloud Storage Static Website
- **Backend**: Cloud Functions (Python 3.12)
- **Database**: Firestore (Native Mode)
- **Infrastructure**: Pulumi 3.0+
- **Deployment**: GitHub Actions

プロジェクトには以下の便利なスクリプトが含まれています：

```
./scripts/test-endpoints.sh

./scripts/test-e2e.sh

./scripts/manage-github-secrets.sh

./scripts/diagnostics.sh

./scripts/deploy-aws.sh
./scripts/deploy-azure.sh
./scripts/deploy-gcp.sh
```

```
make install         # 依存関係をインストール
make build-frontend  # フロントエンドをビルド
```

```
make build-backend   # Lambda パッケージを作成
make test-all        # 全クラウドのデプロイメントをテスト
make deploy-aws      # へデプロイAWS
make pulumi-preview  # 変更プレビューPulumi
make pulumi-up       # スタック適用Pulumi
make clean           # ビルド成果物を削除
```

VS Codeの Dev Containerに対応しています：

```
- Pulumi 3.x
- Node.js 18
- Python 3.12
- AWS CLI, Azure CLI, gcloud CLI
- Docker in Docker

pulumi          # Pulumi CLI
deploy-aws      # 環境にデプロイAWS
deploy-azure    # 環境にデプロイAzure
deploy-gcp      # 環境にデプロイGCP
test-all        # 全エンドポイントテスト
```

システムの健全性をチェック：

```
./scripts/diagnostics.sh
```

- ☒ インストール済みツールの確認
- ☒ クラウドプロバイダー認証状態
- ☒ デプロイメントエンドポイントのテスト
- ☒ Pulumiスタック状態の確認

```
./scripts/test-endpoints.sh

curl https://52z731x570.execute-api.ap-northeast-1.amazonaws.com/
curl https://mcad-staging-api--0000004.livelycoast-fa9d3350.japaneast.azurecontainerapps.io/
curl https://mcad-staging-api-son5b3ml7a-an.a.run.app/
```

```
cd services/frontend
npm install
npm run dev

cd services/backend
pip install -r requirements.txt
uvicorn src.main:app --reload
```

Lambda + API Gatewayの完全自動デプロイメント:

```
cd scripts
./deploy-lambda-aws.sh

PROJECT_NAME=myproject ENVIRONMENT=production ./deploy-lambda-aws.sh
```

スクリプトは以下を自動実行します：

- 依存関係のインストール（manylinux2014_x86_64）
- ZIPパッケージ作成とS3アップロード
- Lambda関数の作成/更新
- API Gateway統合設定
- Lambda権限設定（HTTP API用の正しいSourceArn）
- CloudWatch Logsアクセスログ有効化

エンドポイントの完全なCRUDテスト:

```
./scripts/test-api.sh -e https://YOUR_API_ID.execute-api.ap-northeast-1.amazonaws.com

./scripts/test-api.sh -e https://YOUR_API_ID.execute-api.ap-northeast-1.amazonaws.com --verbose
```

テスト項目：

- ヘルスチェック
- メッセージCRUD操作（作成、取得、更新、削除）
- ページネーション
- エラーハンドリング
- バリデーション

全環境（AWS/GCP/Azure）のエンドツーエンドCRUD動作を検証:

```
./scripts/test-sns-all.sh

./scripts/test-sns-aws.sh
./scripts/test-sns-azure.sh
./scripts/test-sns-gcp.sh
```

テストカバレッジ（2026-02-27 実績）:

- **AWS**: 9 PASS / 0 FAIL
- **Azure**: 17 PASS / 0 FAIL
- **GCP**: 13 PASS / 0 FAIL
- **Total**: 39 PASS / 0 FAIL ☒

テスト項目:

- ☒ Health Checks: 各環境のヘルスエンドポイント検証
- ☒ CRUD Operations: Create/List/Get/Update/Delete
- ☒ Authentication: Cloud-native認証（Cognito/Azure AD/Firebase）
- ☒ Image Upload: PUT /posts with images（AWS S3/Azure Blob/GCP GCS）
- ☒ Error Handling: 404/403/500エラー処理検証

クラウド固有のパス処理:

- AWS/GCP: `/api/messages/`

- Azure: `/api/HttpTrigger/api/messages/`（Flex Consumption対応）

期待される出力例:

```
═══════════════════════════════════════════════════════════════════════

        Multi-Cloud E2E Test Suite═══════════════════════════════════════════════════════════

  Testing: AWS═══════════════════════════════════════════════════════════════════════════✓

 Health check returned 'ok'✓
 Create message (ID: abc123...)✓
 List messages (found 5)✓
 Get message by ID✓
 Update message✓
 Delete message

[GCP/Azure: 同様]═══════════════════════════════════════════════════════════════════════

        Test Summary═══════════════════════════════════════════════════════════════════

Total Tests:  18
Passed:       18
All tests passed! ✓
```

データ永続性検証:

- AWS: DynamoDB (PAY_PER_REQUEST)
- GCP: Firestore (Native Mode)
- Azure: Cosmos DB (Serverless)

包括的な監視とアラートを自動設定:

```
./scripts/setup-monitoring.sh

ALERT_EMAIL=your@email.com ./scripts/setup-monitoring.sh
```

設定内容：

- SNSトピックとメール通知
- Lambda エラー/スロットリング/実行時間/同時実行数アラーム
- API Gateway 5XXエラー/レイテンシアラーム
- DynamoDB スロットリングアラーム

- CloudWatch Logs メトリクスフィルター
- CloudWatch ダッシュボード自動作成

本番運用のために以下のサービス追加を推奨します:

Lambda関数のトレーシング有効化:

```
aws lambda update-function-configuration \
  --function-name YOUR_FUNCTION_NAME \
  --tracing-config Mode=Active
```

FastAPIにX-Ray統合:

```
aws-xray-sdk==2.12.0

from aws_xray_sdk.core import xray_recorder
from aws_xray_sdk.ext.fastapi.middleware import XRayMiddleware

app.add_middleware(XRayMiddleware, recorder=xray_recorder)
```

API Gatewayへの攻撃防御:

```
aws wafv2 create-web-acl \
  --name multicloud-auto-deploy-waf \
  --scope REGIONAL \
  --default-action Allow={} \
  --rules file://waf-rules.json

aws wafv2 associate-web-acl \
  --web-acl-arn YOUR_WEB_ACL_ARN \
  --resource-arn YOUR_API_GATEWAY_ARN
```

プロダクション用ドメイン設定:

```
aws acm request-certificate \
  --domain-name api.yourdomain.com \
  --validation-method DNS

aws apigatewayv2 create-domain-name \
  --domain-name api.yourdomain.com \
  --domain-name-configurations CertificateArn=YOUR_CERT_ARN

aws route53 change-resource-record-sets \
  --hosted-zone-id YOUR_ZONE_ID \
  --change-batch file://route53-changes.json
```

環境変数の安全な管理:

```
aws secretsmanager create-secret \
  --name multicloud-auto-deploy/staging/db-config \
  --secret-string '{"host":"dynamodb","region":"ap-northeast-1"}'
```

```python
import boto3
import json

def get_secret():
    client = boto3.client('secretsmanager')
    response = client.get_secret_value(SecretId='multicloud-auto-deploy/staging/db-config')
    return json.loads(response['SecretString'])
```

共通ライブラリの分離でコールドスタート改善:

```bash
mkdir python
pip install -r requirements.txt -t python/
zip -r layer.zip python/

aws lambda publish-layer-version \
  --layer-name multicloud-auto-deploy-dependencies \
  --zip-file fileb://layer.zip \
  --compatible-runtimes python3.12

aws lambda update-function-configuration \
  --function-name YOUR_FUNCTION_NAME \
  --layers YOUR_LAYER_ARN
```

リクエスト/レスポンスのエッジ処理:

```bash
// CloudFront Function: セキュリティヘッダ追加
function handler(event) {
  var response = event.response;
  response.headers["strict-transport-security"] = {
    value: "max-age=31536000; includeSubdomains",
  };
  response.headers["x-content-type-options"] = { value: "nosniff" };
  response.headers["x-frame-options"] = { value: "DENY" };
  return response;
}
```

DynamoDBの自動バックアップ:

```bash
aws backup create-backup-plan \
  --backup-plan file://backup-plan.json

aws backup create-backup-selection \
  --backup-plan-id YOUR_PLAN_ID \
  --backup-selection file://backup-selection.json
```

問題が発生した場合は トラブルシューティングガイド を参照してください：

- Azure認証問題（Service Principal、Pulumi Provider等）
- GCPリソース競合（State管理、リソースインポート）
- フロントエンドAPI接続問題（ビルド順序、API URL設定）

- 権限エラー（IAM、RBAC設定）

コントリビューションを歓迎します！詳細は CONTRIBUTING.md をご覧ください。

MIT License - 詳細は LICENSE をご覧ください。

- GitHub Actions Documentation
- Terraform Documentation
- Pulumi Documentation

Chapter 2

ソース実装解説書

本ドキュメントは `docs/AI_AGENT_0*.md` の設計書を前提に、実際のソースコードがどのように実装されてい

---

1. プロジェクト全体像
2. バックエンド API（FastAPI）
3. 認証・JWT 検証
4. クラウド別バックエンド実装
5. クラウド別エントリーポイント
6. フロントエンド（React SPA）
7. ローカル開発環境

---

multicloud-auto-deploy は、SNS スタイルの投稿アプリ（文章・画像・タグ）を AWS / Azure / GCP の **3つのクラウドに同時デプロイ** するプラットフォームである。

```
ユーザー

  │
  ├── [AWS]   CloudFront → S3 (React SPA)
  │          API Gateway v2 → Lambda (FastAPI) → DynamoDB
  │
  ├── [Azure] Front Door → Blob Storage (React SPA)
  │          Azure Functions → Cosmos DB
  │
  └── [GCP]   Cloud CDN → GCS (React SPA)
             Cloud Run → Firestore
```

```
services/
  api/                      ← FastAPI バックエンド（クラウド共通）3
    app/
      main.py               ← FastAPI アプリ本体・・ルーター登録CORS
      config.py             ← 環境変数を Pydantic Settings で管理
      models.py             ← Pydantic データモデル
      auth.py               ← JWT 認証ミドルウェア
```

```
      jwt_verifier.py         ← Cognito / Azure AD / Firebase JWT 検証
      backends/
        base.py               ← 抽象基底クラス（インターフェース定義）
        aws_backend.py        ← DynamoDB + S3 実装
        azure_backend.py      ← Cosmos DB + Blob Storage 実装
        gcp_backend.py        ← Firestore + Cloud Storage 実装
        local_backend.py      ← DynamoDB Local + MinIO 実装
      routes/
        posts.py              ← /posts エンドポイント
        profile.py            ← /profile エンドポイント
        uploads.py            ← /uploads エンドポイント
    index.py                  ← AWS Lambda ハンドラー () Mangum
    function_app.py           ← Azure Functions ハンドラー
    function.py               ← GCP Cloud Run ハンドラー
  frontend_react/             ← React 18 + Vite + TypeScript SPA
    src/
      App.tsx                 ← ルーティング定義
      api/                    ← Axios API クライアント
      contexts/AuthContext.←tsx 認証状態管理
      components/             ← UI コンポーネント
      pages/                  ← ページコンポーネント
      config/auth.ts          ← 認証プロバイダー設定
```

次セクション → 2. バックエンド API（FastAPI）

`services/api/app/main.py` が FastAPI アプリの中心。

主な処理の流れ:

```python
try:
    from aws_lambda_powertools import Logger, Metrics, Tracer
    logger = Logger(service="simple-sns-api")
    ...
except ImportError:
    logger = logging.getLogger(__name__)

app = FastAPI(title="Simple SNS API", version="3.0.0", ...)

app.add_middleware(CORSMiddleware, allow_origins=origins, ...)

app.add_middleware(GZipMiddleware, minimum_size=1000)

app.include_router(posts.router)    # /posts
app.include_router(uploads.router)  # /uploads
app.include_router(profile.router)  # /profile
app.include_router(limits.router)   # /limits
```

ライフサイクル管理: `@asynccontextmanager` の `lifespan` 関数でアプリ起動・終了時のログを出力する。

後方互換エイリアス: 旧フロントエンド向けに /api/messages/ エンドポイントを維持している。このエンドポイ
anonymous ユーザーとして扱われる（テスト互換のための意図的な設計）。

---

pydantic_settings.BaseSettings を継承した Settings クラスで全ての環境変数を管理する。

```
class Settings(BaseSettings):
    cloud_provider: CloudProvider = CloudProvider.LOCAL  # "aws"|"azure"|"gcp"|"local"
    auth_disabled: bool = False
    posts_table_name: Optional[str] = None   # 用AWS
    cosmos_db_endpoint: Optional[str] = ...  # 用Azure
    gcp_project_id: Optional[str] = None     # 用GCP
    ...
```

ポイント: - 環境変数名の揺れ（例: COGNITO_USER_POOL_ID vs AWS_COGNITO_USER_POOL_ID）を AliasChoices で吸収している。 - CLOUD_PROVIDER が未設定の場合は、実行環境の特徴から自動検出される（config.py 末尾の auto-detect ロジック）。

クラウド自動検出ロジック:

環境変数	判定クラウド
AWS_EXECUTION_ENV	aws
WEBSITE_INSTANCE_ID	azure
K_SERVICE	gcp
（なし）	local

Pydantic v2 で定義。**camelCase と snake_case 両方を返す**ことが最大の特徴。

```
class Post(BaseModel):
    id: str = Field(..., alias="postId")
    user_id: str = Field(..., alias="userId")
    content: str
    ...

    @model_serializer
    def serialize_model(self) -> dict:
        return {
            # 新フロントエンド用 (camelCase)
            "postId": self.id,
            "userId": self.user_id,
            ...
            # 旧フロントエンド用 (snake_case)
            "id": self.id,
            "author": self.user_id,
            "created_at": self.created_at,
        }
```

`model_config = {"populate_by_name": True}` により、リクエスト受信時は `postId`（alias）でも `id`（フィールド名）のどちらでも受け付ける。

`ListPostsResponse` も同様に `items` / `results` / `messages` の3つのキーで同じデータを返し、複数バージョンのフロン

ファイル	prefix	主な処理
`posts.py`	`/posts`	投稿の CRUD（GET/POST/PUT/DELETE）
`profile.py`	`/profile`	プロフィール取得・更新
`uploads.py`	`/uploads`	画像アップロード用署名付き URL 発行

認証の使い分け:

```
@router.get("")
def list_posts(...) -> ListPostsResponse:
    ...

@router.post("", status_code=201)
def create_post(body: CreatePostBody, user: UserInfo = Depends(require_user)):
    ...
```

`Depends(require_user)` は `auth.py` の関数で、トークンがない場合 HTTP 401 を返す。`Depends(get_current_user)` はトークンなしの場合 `None` を返す（認証任意）。

```
クライアント

  Authorization: Bearer <JWT>
          ↓
  get_current_user() / require_user()
          ↓
  get_jwt_verifier()  ← settings.auth_provider で切替
          ↓
  JWTVerifier.verify_token()
          ↓
  UserInfo { user_id, email, groups }
```

`auth_disabled=true` の場合（ローカル開発時のみ許容）は検証をスキップし、テスト用ユーザーを返す。

UserInfo クラス:

```
@dataclass
class UserInfo:
    user_id: str
    email: Optional[str] = None
    groups: Optional[list[str]] = None

    @property
```

```
    def is_admin(self) -> bool:
        return self.groups is not None and "Admins" in self.groups
```

管理者判定は groups リストに "Admins" が含まれるかで行う。

JWTVerifier クラスが3クラウドの JWT 検証を統一的に処理する。

JWKS エンドポイント:

プロバイダー	JWKS URI
Cognito (AWS)	https://cognito-idp.{region}.amazonaws.com/{pool_id}/.well-known/jwks.json
Firebase (GCP)	https://www.googleapis.com/service_accounts/v1/jwk/securetoken@...
Azure AD	https://login.microsoftonline.com/{tenant_id}/discovery/v2.0/keys

JWKS キャッシュ: 毎リクエストごとに HTTP 取得すると遅延が大きいため、1時間のインメモリキャッシュ

```
def get_jwks(self) -> dict:
    if キャッシュ有効:
        return self._jwks_cache
    # 取得失敗時はキャッシュを使い続ける
    if self._jwks_cache:
        logger.warning("Using expired JWKS cache")
        return self._jwks_cache
    raise
```

トークン検証処理: python-jose ライブラリを使い、JWKS から公開鍵を取得して署名検証 + issuer / audience 検証を行う。検証後 extract_user_info() でクラウドごとの claims 差異を吸収して UserInfo に変換する。

```
@lru_cache(maxsize=1)
def get_backend() -> BackendBase:
    provider = settings.cloud_provider  # 環境変数 CLOUD_PROVIDER
    if provider == CloudProvider.AWS:
        return AwsBackend()
    elif provider == CloudProvider.AZURE:
        return AzureBackend()
    elif provider == CloudProvider.GCP:
        return GcpBackend()
    else:
        return LocalBackend()
```

@lru_cache(maxsize=1) により、同一プロセス内ではバックエンドインスタンスを1回だけ生成する（接続の使い

全バックエンドが実装すべきインターフェースを定義する。

```python
class BackendBase(ABC):
    @abstractmethod
    def list_posts(limit, next_token, tag) -> (list[Post], next_token)
    @abstractmethod
    def get_post(post_id) -> Optional[Post]
    @abstractmethod
    def create_post(body, user) -> dict
    @abstractmethod
    def update_post(post_id, body, user) -> dict   # ※ AWS / Local のみ完全実装
    @abstractmethod
    def delete_post(post_id, user) -> dict
    @abstractmethod
    def get_profile(user_id) -> ProfileResponse
    @abstractmethod
    def update_profile(user, body) -> ProfileResponse
    @abstractmethod
    def generate_upload_urls(count, user, content_types) -> list[dict]
```

データストア: DynamoDB Single Table Design + S3

テーブル設計:

```
PK        SK                          内容
POSTS     <ISO timestamp>#<uuid>      投稿データ
PROFILES  <userId>                    ユーザープロフィール
```

SK に `<timestamp>#<uuid>` を使うことで、`ScanIndexForward=False` クエリが標準の降順（新着順）になる。

投稿一覧取得:

```python
def list_posts(self, limit, next_token, tag):
    query_kwargs = {
        "KeyConditionExpression": "PK = :pk",
        "ExpressionAttributeValues": {":pk": "POSTS"},
        "ScanIndexForward": False,  # 新しい順
        "Limit": limit,
    }
    if next_token:
        query_kwargs["ExclusiveStartKey"] = {"PK": "POSTS", "SK": next_token}
    if tag:
        query_kwargs["FilterExpression"] = "contains(tags, :tag)"
    response = self.table.query(**query_kwargs)
    # next_token = LastEvaluatedKey["SK"]
```

ページネーションには `LastEvaluatedKey["SK"]`（= 最後のアイテムの SK）をトークンとして使う。

画像 URL の解決:

S3 に保存された画像キーは、取得時に `generate_presigned_url`（1時間有効）に変換する。- `https://` で始まる場合はそのまま返す - `http://` はセキュリティ上 Mixed Content となるためスキップする - それ以外（S3 キー）は署名付き URL に変換する

データストア: Cosmos DB（posts / profiles コンテナ） + Blob Storage

DynamoDB との違いは、Cosmos DB では**コンテナ(テーブル)を用途ごとに分ける**点。posts コンテナと profiles コンテナを別々に持つ。

```
self.posts_container = self.database.create_container_if_not_exists(
    id="posts",
    partition_key=PartitionKey(path="/postId"),
)
self.profiles_container = self.database.create_container_if_not_exists(
    id="profiles",
    partition_key=PartitionKey(path="/userId"),
)
```

画像 URL: Blob SAS（24時間有効）に変換。フル Blob URL が既に保存されている場合はキー部分を抽出して SAS を改めて付与する。

インポート安全設計: `azure-cosmos` や `azure-storage-blob` がインストールされていない環境でも `ImportError` をキャッチして警告ログだけ出す。実際に `AzureBackend()` を生成しようとした時に例外を投げる。

データストア: Firestore（posts / profiles コレクション） + Cloud Storage

Firestore ではドキュメント ID でページネーションを行う。

```
def list_posts(self, limit, next_token, tag):
    query = col.order_by("createdAt", direction=DESCENDING).limit(limit + 1)
    if next_token:
        cursor_doc = col.document(next_token).get()
        query = query.start_after(cursor_doc)  # カーソルベースのページネーション
```

`limit + 1` 件取得して `len(docs) > limit` なら次ページがあると判断し、最後のドキュメント ID を `next_token` として返す。

署名付き URL の特殊対応: GCP では Compute Engine のデフォルト認証情報には秘密鍵がないため、`generate_signed_url()` は使えない。代わりに IAM `signBlob` API を使うカスタム実装を行う。認証情報は `google.auth.default()` で取得し、`credentials.valid` を確認してから期限切れ時のみ refresh する（毎リクエストのメタデータサーバー呼び出しを回避）。

データストア: DynamoDB Local + MinIO（S3 互換）

AWS バックエンドと同じ Single Table Design を使うことで、ローカルで発見したバグが AWS でも同様に再現することを保証している。テーブルが存在しない場合は**初回起動時に自動作成**する。

```python
def _init_dynamodb(self):
    # テーブルが存在しなければ作成
    try:
        self.table = self.dynamodb.create_table(
            TableName=table_name,
            KeySchema=[{"AttributeName": "PK", ...}, {"AttributeName": "SK", ...}],
            ...
        )
    except self.dynamodb.meta.client.exceptions.ResourceInUseException:
        self.table = self.dynamodb.Table(table_name)  # 既存テーブルを使う
```

FastAPI アプリ本体（`app/main.py`）は3クラウドで**共通**。クラウドごとにその前段となるアダプター層が異な

```python
from mangum import Mangum
from app.main import app

handler = Mangum(app, lifespan="off")
```

`Mangum` が Lambda イベント（API Gateway Proxy v2 形式）を ASGI に変換し、FastAPI へ渡す。`lifespan="off"` は Lambda の短命実行モデルに対応するための設定。

Lambda Layer に FastAPI 等の依存ライブラリを格納し、アプリコード（〜78KB）と別管理することでデプ

Azure Functions には Mangum 相当の標準アダプターがないため、**カスタム ASGI ブリッジ**を自前実装している。

主な処理フロー:

```
Azure Functions HttpRequest
    ↓
route_params["route"] からパスを取得
    "HttpTrigger/" プレフィックスを除去
    ↓
ASGI scope dict を組み立て
    {type: "http", path: "/posts", method: "GET", ...}
    ↓
FastAPI ASGI app を呼び出し
    ↓レスポンスを
 func.HttpResponse に変換して返す
```

CORS Preflight（OPTIONS） は Azure Functions レベルで直接処理し、FastAPI に渡さずに 204 を返す（速度・安定性のため）。

インポートエラーが発生しても 503 でエラー内容を返すフェイルセーフ設計：

```python
_IMPORT_ERROR = None
try:
    from app.main import app as fastapi_app
```

```
except Exception as e:
    _IMPORT_ERROR = traceback.format_exc()
...
if fastapi_app is None:
    return func.HttpResponse(
        body=json.dumps({"error": "Service unavailable", "detail": _IMPORT_ERROR}),
        status_code=503,
    )
```

Cloud Run は通常の HTTP サーバーとして動作するため、uvicorn でそのまま FastAPI を起動できる。Lambda や Azure Functions のような特殊なアダプターは必要ない。

Dockerfile で CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"] のような起動コマンドを指定するだけで Cloud Run 上で動作する。

---

services/frontend_react/ に React 18 + Vite + TypeScript + Tailwind CSS で構築された SPA。3クラウドすべてで**同一のビルド成果物**をオブジェクトストレージ（S3 / Blob Storage / GCS）に配置し、CDN で配信する。

vite.config.ts でビルド時のベースパスを /sns/ に設定している（CDN の /sns/* パスにマッピングされるため）。

```
<AuthProvider>
  <Layout>
    <Routes>
      <Route path="/"              element={<HomePage />} />
      <Route path="/post/:postId"  element={<PostPage />} />
      <Route path="/profile"       element={<ProfilePage />} />
      <Route path="/login"         element={<LoginPage />} />
      <Route path="/auth/callback" element={<CallbackPage />} />
      <Route path="*"              element={<HomePage />} />
    </Routes>
  </Layout>
</AuthProvider>
```

AuthProvider が認証状態をアプリ全体に提供する（React Context パターン）。

```
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL ?? '',  // 空文字 → Vite プロキシ経由でローカル API
  timeout: 15000,
});

// 全リクエストに JWT トークンを付与
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('id_token') || localStorage.getItem('access_token');
```

```
  if (token) config.headers['Authorization'] = `Bearer ${token}`;
  return config;
});
```

Cognito は id_token（audience が client_id と一致する）を優先して使う。access_token はフォールバック。

ビルド時の環境変数 VITE_AUTH_PROVIDER で認証プロバイダーを切り替える。

VITE_AUTH_PROVIDER	認証方式
aws	Cognito PKCE 認証コードフロー
azure	Azure AD 暗黙的フロー（id_token フラグメント）
firebase	Firebase SDK Google Sign-In（ポップアップ）
none	認証なし（ローカル開発）

AWS (Cognito) の PKCE フロー: 1. generateCodeVerifier() でランダム文字列生成 → sessionStorage に保存 2. SHA-256 ハッシュ → base64url エンコード → code_challenge 3. Cognito 認証エンドポイントへリダイレクト 4. コールバック（/auth/callback）で code を受取り + code_verifier で検証 → token 取得

---

localStorage にトークンを保存・読込し、window の storage イベントを監視することでタブ間でログイン状態を

```
// タブ間同期
useEffect(() => {
  const onStorage = () => setToken(readToken());
  window.addEventListener("storage", onStorage);
  return () => window.removeEventListener("storage", onStorage);
}, []);
```

コンポーネント	役割
HomePage	投稿一覧表示・タグフィルタ・キーワード検索・ページネーション（“もっと見る”）
PostForm	投稿作成フォーム（テキスト・マークダウン・画像複数枚・タグ）
PostCard	1件の投稿表示（マークダウン対応・画像ライトボックス）
ProfilePage	ニックネーム・自己紹介の表示と編集
LoginPage	各クラウドの認証フローへのリダイレクト
CallbackPage	Cognito / Azure AD の認証コールバック処理・token 保存

```
1. POST /uploads/presigned-urls  →  { urls: [{uploadUrl, key}] }
2. 各 uploadUrl に （ブラウザから直接PUT S3/Blob/GCS へアップロード）
3. POST /posts { imageKeys: [key1, key2, ...] }  →  投稿作成
```

バックエンドを経由しないため、大容量画像でもバックエンドに負荷をかけない設計。アップロード上限は `GET /limits` で取得し、ハードコードしない。

`docker-compose.yml` で以下の4サービスを起動する。

サービス	ポート	用途
`api`	8000	FastAPI（ホットリロード付き uvicorn）
`frontend_web`	8080	Python SSR フロントエンド（レガシー）
`dynamodb-local`	8001	DynamoDB Local（AWS 互換）
`minio`	9000/9001	MinIO（S3 互換オブジェクトストレージ）

API サービスの環境変数（ローカル）：

```
CLOUD_PROVIDER=local
AUTH_DISABLED=true      # ローカルのみ許容
DYNAMODB_ENDPOINT=http://dynamodb-local:8000
DYNAMODB_TABLE_NAME=simple-sns-local
MINIO_ENDPOINT=http://minio:9000
MINIO_PUBLIC_ENDPOINT=http://localhost:9000   # ブラウザからの直接アクセス用
```

`MINIO_ENDPOINT`（コンテナ内部 URL）と `MINIO_PUBLIC_ENDPOINT`（ブラウザからの URL）を分けることで、コンテナ内→MinIO の通信とブラウザ→MinIO の通信が両立する。

```
cd /workspaces/multicloud-auto-deploy
docker compose up -d

curl http://localhost:8000/health

open http://localhost:8000/docs

open http://localhost:9001  # minioadmin / minioadmin
```

`services/api/app` ディレクトリが `docker-compose.yml` でコンテナにマウントされるため、ソース変更は即座にホッ

`LocalBackend._init_dynamodb()` が初回起動時に DynamoDB Local へテーブルを自動作成する。手動でのテーブル作成は不要。

MinIO のバケットも同様に `LocalBackend._init_storage()` 内で自動作成する。

パターン	実装箇所	説明
Strategy パターン	`backends/`	`BackendBase` を実装するクラスをクラウドごとに差し替え
Factory + LRU Cache	`backends/__init__.py`	`get_backend()` で1回だけインスタンスを生成
Pydantic Settings	`config.py`	環境変数を型安全に管理、`AliasChoices` で名前揺れを吸収
Dual serialization	`models.py`	camelCase と snake_case を同時に返してフロントエンド互換
JWKS キャッシュ	`jwt_verifier.py`	1時間キャッシュ、失敗時は期限切れキャッシュをフォールバック
Presigned URL	各バックエンド	画像をバックエンド経由せずブラウザから直接アップロード
PKCE フロー	`config/auth.ts`	AWS Cognito で認証コードフローを安全に実装

Chapter 3

インフラ・CI/CD解説書

> 本ドキュメントは `services/api/`・`services/frontend_react/` のアプリコードを解説した IMPLEMENTATION_GUIDE.md の**姉妹ドキュメント**である。 ここでは **インフラ (Pulumi)・CI/CD (GitHub Actions)・スクリプト** のソースを扱う。

---

1. インフラ全体の考え方
2. Pulumi AWS (`infrastructure/pulumi/aws/__main__.py`)
3. Pulumi Azure (`infrastructure/pulumi/azure/__main__.py`)
4. Pulumi GCP (`infrastructure/pulumi/gcp/__main__.py`)
5. CI/CD ワークフロー (GitHub Actions)
6. フロントエンド SPA デプロイワークフロー
7. スクリプト群 (`scripts/`)

Terraform ではなく **Pulumi** を使う理由は、Python コードで条件分岐・ループ・関数を自由に使えるため
`Pulumi.<stack>.yaml` に記述し、デプロイ状態は **Pulumi Cloud**（リモートステート）で管理する。

スタック名	ブランチ	用途
`staging`	`develop`	自動デプロイ・動作確認用
`production`	`main`	本番環境（即時反映）

各スタックは独立したクラウドリソースを持つ（S3　バケット名・Lambda　関数名などに `{stack}` サフィックスが付く）。

```
project_name = "multicloud-auto-deploy"
stack = pulumi.get_stack()   # "staging" または "production"

f"{project_name}-{stack}-api"
```

全リソースに `common_tags`（Project / ManagedBy / Environment）を付与してコスト管理・検索を容易にする

ファイル: infrastructure/pulumi/aws/**main**.py（約965行）

```
IAM Role (Lambda 用)
   └── AWSLambdaBasicExecutionRole ポリシーアタッチ
   └── DynamoDB + S3 アクセス用インラインポリシー

Cognito User Pool
   └── User Pool Client (PKCE コードフロー)
   └── User Pool Domain 認証ホスト( UI)

DynamoDB Table (Single Table Design)
   ├── PK="PK" / SK="SK"
   ├── GSI: PostIdIndex (hash=postId)
   └── GSI: UserPostsIndex (hash=userId, range=createdAt)

S3 Bucket: （画像アップロード先）images
   └── CORS 設定（ブラウザから直接 PUT できるよう）
   └── （PublicAccessBlockpublic 禁止）

Lambda （LayerFastAPI / Mangum / Pydantic 等）
   └── services/api/lambda-layer.zip から生成

Lambda （FunctionFastAPI + ) Mangum
   └── ENV: POSTS_TABLE_NAME, IMAGES_BUCKET_NAME, COGNITO_* 等

API Gateway v2 (HTTP API)
   └── $default ルート → Lambda 統合

S3 Bucket: （frontendReact SPA 配信）
   └── OAC (Origin Access Control) で CloudFront のみ許可

CloudFront Distribution
   ├── /sns/* → S3 frontend / CloudFront Function で /sns → /sns/index.html リライト
   └── /*     → S3 frontend (landing pages)
```

```python
user_pool_client = aws.cognito.UserPoolClient(
    allowed_oauth_flows=["code"],         # PKCE コードフロー
    allowed_oauth_scopes=["openid", "email", "profile"],
    callback_urls=(
        ([f"https://{cf_domain}/sns/auth/callback"] if cf_domain else [])
        + ([f"https://{custom_domain}/sns/auth/callback"] if custom_domain else [])
    ),
    access_token_validity=1,   # 時間1
    id_token_validity=1,       # 時間1
    refresh_token_validity=30, # 日30
)
```

`cf_domain`（CloudFront ドメイン）と `custom_domain`（カスタムドメイン）の両方を登録することで、どちらの CI とローカルでは ZIP のパスが異なるため、`GITHUB_WORKSPACE` 環境変数で分岐している。

```
workspace_root = os.environ.get("GITHUB_WORKSPACE")
if workspace_root:
    layer_zip_path = pathlib.Path(workspace_root) / "services/api/lambda-layer.zip"
else:
    # __file__ から階層上がリポジトリルート3
    layer_zip_path = pathlib.Path(__file__).parent.parent.parent /
    "services/api/lambda-layer.zip"
```

ZIP が存在しない場合はウォーニングを出してスキップする（初回デプロイ時など）。

React SPA は HTML5 History API でルーティングするため、CDN 側で /sns → /sns/index.html のリライトが必要。CloudFront Function を Pulumi のコード内にインラインで定義している。

```
spa_rewrite_function = aws.cloudfront.Function(
    "spa-sns-rewrite",
    name=f"spa-sns-rewrite-{stack}",
    runtime="cloudfront-js-2.0",
    code="""
function handler(event) {
    var uri = event.request.uri;
    if (uri === '/sns' || uri === '/sns/') {
        event.request.uri = '/sns/index.html';
    } else if (uri.startsWith('/sns/') && !uri.includes('.')) {
        event.request.uri = '/sns/index.html';
    }
    return event.request;
}
""",
)
```

ファイル: infrastructure/pulumi/azure/**main**.py（約612行）

```
Resource Group (japaneast)
   ├── Storage Account: mcadfunc{suffix}  ← Azure Functions 用
   ├── Storage Account: mcadweb{suffix}   ← フロントエンド静的サイト用
   |     └── $web コンテナ（静的 Web サイトホスティング）
   ├── Application （監視）Insights
   ├── Cosmos DB アカウント
   |     └── コンテナ: posts / profiles
   ├── Azure AD アプリ登録（認証）
   |     └── リダイレクト URI: https://{frontend_domain}/sns/auth/callback
   ├── Azure （FunctionsFlex ) Consumption
   |     └── HTTP Trigger: {*route} → FastAPI
   └── Azure Front （) DoorCDN
         ├── /sns/* → Blob Storage $web/sns/ (React SPA)
         └── /*     → Blob Storage $web (landing pages)
```

Azure ストレージアカウント名はグローバル一意が必要なため pulumi_random で6文字サフィックスを生成する。同じインスタンスを2つのアカウントで共有することで、識別が容易にな

```
storage_suffix = random.RandomString("storage-suffix", length=6, special=False, upper=False)

functions_storage_name = storage_suffix.result.apply(lambda s: f"mcadfunc{s}")
frontend_storage_name  = storage_suffix.result.apply(lambda s: f"mcadweb{s}")
```

AWS Cognito が PKCE コードフローを使うのに対し、Azure AD は**暗黙的フロー**（id_token フラグメント）を使う。

```
app_registration = azuread.Application(
    "app-registration",
    web=azuread.ApplicationWebArgs(
        redirect_uris=[f"https://{frontend_domain}/sns/auth/callback"],
        implicit_grant=azuread.ApplicationWebImplicitGrantArgs(
            id_token_issuance_enabled=True,   # 暗黙的フロー
        ),
    ),
)
```

フロントエンドのコールバックハンドラーは URL フラグメント（#id_token=...）を解析してトークンを取得す

AFD の SpaRuleSet URL リライトルールで /sns/* の非アセットパスを /sns/index.html にリライトし、React SPA のクライアントルーティングを実現する。

ファイル: infrastructure/pulumi/gcp/**main**.py（約585行）

```
GCP API 有効化（Cloud Run / Secret Manager / Firebase 等）
Firebase プロジェクト・Web アプリ登録
Cloud Audit （全サービス対象）Logs

GCS バケット: frontend   ← React SPA + landing pages
GCS バケット: uploads    ← 画像アップロード

Secret （環境変数を秘密として管理）Manager

Cloud Run: api           ← FastAPI (uvicorn)
  └── サービスアカウント（Firestore / GCS / Secret Manager 権限）

Cloud （CDNClassic External ）LB
  └── /* → GCS バケット（Backend ）Bucket
```

```
services = ["run.googleapis.com", "secretmanager.googleapis.com", "firebase.googleapis.com",
    ...]
enabled_services = [gcp.projects.Service(f"enable-{s}", ...) for s in services]

firebase_project = gcp.firebase.Project(
    "firebase-project",
    opts=pulumi.ResourceOptions(depends_on=enabled_services),
```

```
)
```

PulumiはCloud Runの**サービス設定**（メモリ・環境変数・サービスアカウント）を管理するが、**コンテナ**
CI/CDワークフロー（`gcloud run deploy`）が担う。Pulumiデプロイ時にコンテナイメージを要求するという制

Firebaseの認証プロバイダー設定（Googleサインイン）はOAuth同意画面の設定が必要で
Pulumiから自動化できない。コードにコメントで明示している。

ファイル	役割
`deploy-aws.yml`	AWS全リソース（Pulumi + Lambda + フロントエンド含む）
`deploy-azure.yml`	Azure全リソース（Pulumi + Functions + フロントエンド含む）
`deploy-gcp.yml`	GCP全リソース（Pulumi + Cloud Run + フロントエンド含む）
`deploy-frontend-web-aws.yml`	React SPAをS3/CloudFront にのみ再デプロイ（差分更新）
`deploy-frontend-web-azure.yml`	React SPAをBlob Storage/AFDにのみ再デプロイ
`deploy-frontend-web-gcp.yml`	React SPAをGCS/CDNにのみ再デプロイ
`deploy-landing-*.yml`	ランディングページ（静的HTML）の各クラウドへの配信
`version-bump.yml`	`versions.json`のバージョン自動更新

3つのワークフローすべてで同じルールを採用している。

```
environment: >-
  ${{ github.event_name == 'workflow_dispatch'
        && github.event.inputs.environment
        || (github.ref == 'refs/heads/main' && 'production' || 'staging') }}
```

Pulumi は同一スタックへの並行デプロイで `ResourceConflictException` / `409 Conflict` が発生するため、`concurrency`で直列化している。

```
concurrency:
  group: deploy-aws-${{ ... }}
  cancel-in-progress:
    false # AWS/GCP: 実行中のジョブはキャンセルしない
    # Azure staging: （コスト削減のため古いtrue Run をキャンセル）
```

```
1. AWS 認証（aws-actions/configure-aws-）credentials
2. Pulumi CLI インストール
3. スタック名の決定（→developstaging / →）mainproduction
```

```
4. .github/config/aws.{stack}.env から設定を読込（single source of ）truth
5. Pulumi スタック選択 / 初期化
6. Pulumi （インフラ変更適用）up
7. Pulumi outputs 取得（Lambda 名・バケット名・Cognito 設定等）
8. Lambda Layer ZIP ビルド（scripts/build-lambda-layer.）sh
9. Lambda 関数コードの ZIP 化・アップロード（aws lambda update-function-）code
10. CloudFront エイリアス + ACM 証明書の確認・修正（safety ）net
11. Cognito コールバック URL の確認・修正（safety ）net
12. React SPA ビルド（npm run ）build
13. S3 同期（aws s3 ）sync
14. CloudFront キャッシュ無効化（aws cloudfront create-）invalidation
```

「Safety net」ステップの目的: Pulumiがカスタムドメイン設定を誤って消してしまうことがあるため、デ AWS CLI で直接確認・修正するステップを追加している。Pulumiの宣言的な制御と、より強制力のある命令的な確認の二重構造になっている。

設定ファイルの一元管理: `Pulumi.*.yaml`はgitignoreされておりリモートで使えないため、代わりに `.github/config/aws.staging.env` 等のコミット済みファイルに設定値を記述する。CIはこのファイルを `source` して環境変数を読み込む。

```
1. （PulumiAzure インフラ）
2. Azure Functions 用パッケージビルド
   - linux/amd64 Docker コンテナで pip （installARM ホストとのアーキテクチャ差異を回避）
   - ZIP 化して Azure CLI でアップロード（az functionapp deployment source config-）zip
3. az storage blob service-properties update で静的 Web サイトホスティング有効化
4. React SPA ビルド・Blob Storage アップロード（az storage blob upload-）batch
5. AFD キャッシュパージ（az afd endpoint ）purge
```

ARM → AMD64 クロスビルド: GitHub Actions のランナーは ARM (aarch64) のホストで動作する場合があるため、Lambda / Functions 用の Python パッケージは `--platform linux/amd64` の Docker コンテナ内でインストールする。

```
docker run --rm --platform linux/amd64 \
  -v "$(pwd)/package:/package" \
  python:3.12-slim \
  bash -c "pip install -r /package/requirements.txt -t /package"
```

```
1. google-github-actions/auth で認証（GCP_CREDENTIALS シークレット）
2. （PulumiGCP インフラ）
3. Docker イメージビルド・Artifact Registry プッシュ
4. Cloud Run サービス更新（gcloud run ）deploy
5. React SPA ビルド・GCS アップロード（gsutil ）rsync
6. CDN キャッシュ無効化（gcloud compute url-maps invalidate-cdn-）cache
```

`deploy-frontend-web-*.yml`はAPIやインフラを触らずに**フロントエンドだけを更新**したい場合に使う差分更新用 SPAのファイルを変更した際にPulumiを再実行せずに済むため、デプロイ時間を短縮できる。

```
1. Pulumi outputs 取得（バケット名・Cognito 設定・CloudFront ）ID
   ← インフラには触れず読み取りのみ
2. npm ci / npm run build
   - VITE_API_URL / VITE_AUTH_PROVIDER=aws / VITE_COGNITO_* を環境変数として注入
3. S3 同期（差分更新）
   - アセット類（JS/）CSS: cache-control max-age=31536000, immutable
   - index.html のみ: max-age（ブラウザキャッシュを短くして更新を即反映）=300
4. Cognito コールバック URL を aws cognito-idp update-user-pool-client で同期
   - CloudFront ドメイン + カスタムドメインの両方を登録
5. CloudFront キャッシュ無効化（/sns）/*
```

キャッシュ戦略の意図: ハッシュ付きアセット（`index-abc123.js` 等）は Vite がファイル名を変えるため長期キャッシュ可。一方 `index.html` は常に最新のアセット名を参照するため短期キャッシュにする。

```
1. Pulumi outputs 取得（ストレージアカウント名・AFD エンドポイント）
2. npm run （buildVITE_AUTH_PROVIDER=azure 等を注入）
3. az storage blob upload-batch で $web/sns/ へアップロード
4. az afd endpoint purge でキャッシュパージ
```

```
1. Pulumi outputs 取得（GCS バケット名）
2. npm run （buildVITE_AUTH_PROVIDER=firebase 等を注入）
3. gsutil -m rsync -r dist/ gs://{bucket}/sns/
4. gcloud compute url-maps invalidate-cdn-cache
```

Lambda Layer を `linux/amd64` Docker コンテナでビルドして ZIP 化するスクリプト。デプロイホストが ARM (Apple M1/M2 / aarch64) でも正しいアーキテクチャのバイナリを生成できる。

```
1. requirements-layer.txt を参照
2. docker run --platform linux/amd64 python:3.12-slim で pip install
3. python/ 以下に配置して ZIP 化 → services/api/lambda-layer.zip
```

Layer には `fastapi`, `mangum`, `pydantic`, `python-jose` 等が含まれる。 `boto3` / `botocore` は Lambda ランタイムに同梱されているため**除外**し、Layer サイズを削減している。

各クラウドに手動デプロイするためのラッパースクリプト。主に開発時の動作確認に使う。CI/CD ワークフロー（`.github/workflows/*.yml`）と同等の処理を行うが、インタラクティブな確認ステップを含む。

スクリプト	内容
`test-api.sh`	各クラウドの API エンドポイントに curl でリクエストし、ステータスコードを確認
`test-e2e.sh`	全クラウドを対象とした E2E テスト（投稿作成→取得→削除）
`test-sns-aws.sh` 等	クラウド個別の SNS 機能テスト
`test-auth-crud.sh`	認証付き CRUD 操作のテスト
`diagnose-environments.sh`	各クラウドの設定値・エンドポイントの診断

`versions.json` のバージョン番号をインクリメントするスクリプト。`version-bump.yml` ワークフローから呼び出される。セマンティックバージョニング（major/minor/patch）に対応。

原則	実装箇所	説明
IaC 一元管理	`infrastructure/pulumi/`	クラウドリソースはすべて Pulumi Python コードで定義
設定の単一ソース	`.github/config/*.env`	`Pulumi.*.yaml` は gitignore されるため、CI 用設定ファイルをリポジトリにコミット
Safety net	`deploy-aws.yml`	Pulumi だけに頼らず AWS CLI で CloudFront / Cognito を事後確認・修正
ARM/AMD64 分離	CI ワークフロー	Python パッケージは常に `linux/amd64` Docker でビルド
キャッシュ戦略	`deploy-frontend-web-*.yml`	ハッシュ付きアセット: 永続キャッシュ / `index.html`: 短期キャッシュ
差分デプロイ	`deploy-frontend-web-*.yml`	フロントエンド変更時は Pulumi を再実行しない
同時実行防止	`concurrency:` ブロック	Pulumi の競合エラーを防ぐためスタック単位で直列化

Chapter 4

カスタムドメイン設定

Steps for configuring custom domains when using different domains per cloud.

All 3 cloud custom domains are operational.

Cloud	Custom Domain	Target Endpoint	Status
AWS	`www.aws.ashnova.jp`	`d1qob7569mn5nw.cloudfront.net`	☒ HTTPS active (directly fixed 2026-02-21)
Azure	`www.azure.ashnova.jp`	`nwaj-production-diev0w-f9ekdmehb0bgaaaw.z01.azurefd.net`	☒ HTTPS active ⚠ /sns/* needs investigation
GCP	`www.gcp.ashnova.jp`	`34.8.38.222` (A record)	☒ HTTPS active

```
curl -I https://www.aws.ashnova.jp        # 200 OK
curl -I https://www.azure.ashnova.jp      # 200 OK
curl -I https://www.gcp.ashnova.jp        # 200 OK

curl https://www.aws.ashnova.jp/health    # 200 {"status":"healthy"}
curl https://www.gcp.ashnova.jp/health    # 200 {"status":"healthy"}
```

AWS

- ☒ ACM certificate `ISSUED` confirmed → `arn:aws:acm:us-east-1:278280499340:certificate/914b86b1-4c10-4354-91cf-19` (expires 2027-03-12)
- ☒ DNS: CNAME `www.aws.ashnova.jp` → `d1qob7569mn5nw.cloudfront.net` configured
- ☒ CloudFront `E214XONKTXJEJD` alias + ACM certificate set directly (fixed via AWS CLI on 2026-02-21)
  - Issue: `pulumi up` did not apply alias/certificate (remained on CloudFrontDefaultCertificate)
  - Fix: Set alias `www.aws.ashnova.jp` + cert `914b86b1` via `aws cloudfront update-distribution`

- [!] **Note before re-running Pulumi**: Must set `pulumi config set customDomain www.aws.ashnova.jp --stack production` and `pulumi config set acmCertificateArn arn:aws:acm:us-east-1:278280499340:certificate/914b86b1-4c10-4354-91cf-19c4460dcde5 --stack production`

☒ CORS updated

Azure

☒ `az afd custom-domain create` executed (`azure-ashnova-jp`)
☒ Custom domain attached to both routes
☒ DNS: TXT record `_dnsauth.www.azure.ashnova.jp` configured (verified)
☒ DNS: CNAME `www.azure.ashnova.jp` → `mcad-production-diev0w-f9ekdmehb0bga5aw.z01.azurefd.net` configured
☒ Managed Certificate issued, HTTPS active

- [⚠] `/sns/*` intermittent 502 issue → under investigation (see AZURE_SNS_FIX_REPORT.md)

GCP

☒ Pulumi config set (`customDomain: www.gcp.ashnova.jp`)
☒ `pulumi up --stack production` completed
☒ DNS: A record `www.gcp.ashnova.jp` → `34.8.38.222` configured
☒ Managed SSL certificate `ACTIVE` confirmed

Cloud	Custom Domain	Target Endpoint
AWS	`www.aws.ashnova.jp`	`d1qob7569mn5nw.cloudfront.net`
Azure	`www.azure.ashnova.jp`	`mcad-production-diev0w-f9ekdmehb0bga5aw.z01.azurefd.net`
GCP	`www.gcp.ashnova.jp`	`34.8.38.222` (A record)

Cloud	Type	Current Endpoint	Distribution ID
AWS	CloudFront	`d1tf3uumcm4bo1.cloudfront.net`	E1TBH4R432SZBZ
Azure	Front Door	`mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net`	mcad-staging-d45ihd
GCP	Cloud CDN	`34.117.111.182` (IP address)	-

Cloud	Type	Current Endpoint	Distribution ID
AWS	CloudFront	`d1qob7569mn5nw.cloudfront.net`	E214XONKTXJEJD

Cloud	Type	Current Endpoint	Distribution ID
Azure	Front Door	`mcad-production-diev0w-f9ekdmehb0bga5aw.z01.azurefd.net`	mcad-production-diev0w
GCP	Cloud CDN	`34.8.38.222` (IP address)	-

Custom domains configured for this project:

- **AWS**: `www.aws.ashnova.jp`
- **Azure**: `www.azure.ashnova.jp`
- **GCP**: `www.gcp.ashnova.jp`

> Note: Generic procedures use placeholders such as `aws.yourdomain.com`. In practice, use the ashnova.jp domains above.

- Domain ownership verified
- AWS Route 53 (recommended) or external DNS provider

```
aws acm request-certificate \
  --domain-name www.aws.ashnova.jp \
  --validation-method DNS \
  --region us-east-1

CERT_ARN=$(aws acm list-certificates \
  --region us-east-1 \
  --query "CertificateSummaryList[?DomainName=='www.aws.ashnova.jp'].CertificateArn" \
  --output text)

echo "Certificate ARN: $CERT_ARN"
```

```
aws acm describe-certificate \
  --certificate-arn $CERT_ARN \
  --region us-east-1 \
  --query 'Certificate.DomainValidationOptions[0].ResourceRecord'
```

Configure in your DNS provider:

- Record type: `CNAME`
- Name: `_abc123.www.aws.ashnova.jp`
- Value: `_xyz456.acm-validations.aws.`

Modify the CloudFront Distribution section in `infrastructure/pulumi/aws/__main__.py`:

```
cert_arn = config.get("acmCertificateArn")  # Retrieved from Pulumi config
custom_domain = config.get("customDomain")  # e.g. aws.yourdomain.com
```

```

cloudfront_kwargs = {
    # ... existing config ...
    "aliases": [custom_domain] if custom_domain else [],
    "viewer_certificate": aws.cloudfront.DistributionViewerCertificateArgs(
        acm_certificate_arn=cert_arn,
        ssl_support_method="sni-only",
        minimum_protocol_version="TLSv1.2_2021",
    ) if cert_arn else aws.cloudfront.DistributionViewerCertificateArgs(
        cloudfront_default_certificate=True,
    ),
    # ... remaining config ...
}
```

```
cd infrastructure/pulumi/aws
pulumi config set customDomain www.aws.ashnova.jp --stack production
pulumi config set acmCertificateArn arn:aws:acm:us-east-1:ACCOUNT_ID:certificate/CERT_ID
    --stack production
pulumi up --stack production
```

Verify the CloudFront domain for your Pulumi environment (production/staging):

```
cd infrastructure/pulumi/aws
pulumi stack select production  # or staging
CLOUDFRONT_DOMAIN=$(pulumi stack output cloudfront_domain)
echo "CloudFront Domain: $CLOUDFRONT_DOMAIN"
```

For Route 53:

```
aws route53 change-resource-record-sets \
  --hosted-zone-id YOUR_ZONE_ID \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name": "www.aws.ashnova.jp",
        "Type": "CNAME",
        "TTL": 300,
        "ResourceRecords": [{"Value": "d1qob7569mn5nw.cloudfront.net"}]
      }
    }]
  }'
```

For external DNS providers:

- Record type: `CNAME`
- Name: `www.aws.ashnova.jp`
- Value:
  - Production: `d1qob7569mn5nw.cloudfront.net`
  - Staging: `d1tf3uumcm4bo1.cloudfront.net`

- Domain ownership verified
- Azure DNS (recommended) or external DNS provider

Retrieve environment resource info:

```
cd infrastructure/pulumi/azure
pulumi stack select production  # or staging
FRONTDOOR_HOSTNAME=$(pulumi stack output frontdoor_hostname)
FRONTDOOR_PROFILE=$(pulumi stack output frontdoor_profile_name)
FRONTDOOR_ENDPOINT=$(pulumi stack output frontdoor_endpoint_name)
RESOURCE_GROUP=$(pulumi stack output resource_group_name)

echo "Front Door Hostname: $FRONTDOOR_HOSTNAME"
echo "Profile Name: $FRONTDOOR_PROFILE"
```

Create custom domain:

```
ENVIRONMENT="production"
RESOURCE_GROUP="multicloud-auto-deploy-${ENVIRONMENT}-rg"
PROFILE_NAME="multicloud-auto-deploy-${ENVIRONMENT}-fd"
CUSTOM_DOMAIN_NAME="azure-ashnova-jp"
HOSTNAME="www.azure.ashnova.jp"

az afd custom-domain create \
  --resource-group $RESOURCE_GROUP \
  --profile-name $PROFILE_NAME \
  --custom-domain-name $CUSTOM_DOMAIN_NAME \
  --host-name $HOSTNAME \
  --certificate-type ManagedCertificate
```

```
az afd custom-domain show \
  --resource-group $RESOURCE_GROUP \
  --profile-name $PROFILE_NAME \
  --custom-domain-name $CUSTOM_DOMAIN_NAME \
  --query "validationProperties"
```

Configure in your DNS provider:

- Record type: `TXT`
- Name: `_dnsauth.www.azure.ashnova.jp`
- Value: `abc123def456` (validationToken)

```
ENDPOINT_NAME=$(pulumi stack output frontdoor_endpoint_name)
echo "Endpoint Name: $ENDPOINT_NAME"

az afd route create \
  --resource-group $RESOURCE_GROUP \
  --profile-name $PROFILE_NAME \
```

```
  --endpoint-name $ENDPOINT_NAME \
  --route-name custom-domain-route \
  --origin-group-name default-origin-group \
  --supported-protocols Https \
  --custom-domains $CUSTOM_DOMAIN_NAME \
  --forwarding-protocol HttpsOnly \
  --https-redirect Enabled
```

Verify the Front Door Hostname:

```
cd infrastructure/pulumi/azure
pulumi stack select production  # or staging
FRONTDOOR_HOSTNAME=$(pulumi stack output frontdoor_hostname)
echo "Front Door Hostname: $FRONTDOOR_HOSTNAME"
```

For Azure DNS:

```
az network dns record-set cname set-record \
  --resource-group YOUR_DNS_RG \
  --zone-name ashnova.jp \
  --record-set-name www.azure \
  --cname $FRONTDOOR_HOSTNAME
```

For external DNS providers:

- Record type: `CNAME`
- Name: `www.azure.ashnova.jp`
- Value:
  - Production: `mcad-production-diev0w-f9ekdmehb0bga5aw.z01.azurefd.net`
  - Staging: `mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net`

```
az afd custom-domain show \
  --resource-group $RESOURCE_GROUP \
  --profile-name $PROFILE_NAME \
  --custom-domain-name $CUSTOM_DOMAIN_NAME \
  --query "{provisioningState, domainValidationState, deploymentStatus}"
```

- Domain ownership verified
- Google Cloud DNS (recommended) or external DNS provider

Modify the SSL certificate section in `infrastructure/pulumi/gcp/__main__.py`:

```
custom_domain = config.get("customDomain")  # e.g. gcp.yourdomain.com

managed_ssl_cert = gcp.compute.ManagedSslCertificate(
    f"{project_name}-{stack}-ssl-cert",
    managed=gcp.compute.ManagedSslCertificateManagedArgs(
        domains=[custom_domain] if custom_domain else ["example.com"],
```

```
    ),
    opts=pulumi.ResourceOptions(
        delete_before_replace=True,
    ),
)
```

```
cd infrastructure/pulumi/gcp
pulumi config set customDomain www.gcp.ashnova.jp --stack production
pulumi up --stack production
```

Note: Managed SSL certificate provisioning can take up to 60 minutes.

Verify the CDN IP address:

```
cd infrastructure/pulumi/gcp
pulumi stack select production  # or staging
CDN_IP=$(pulumi stack output cdn_ip_address)
echo "CDN IP Address: $CDN_IP"
```

For Google Cloud DNS:

```
gcloud dns record-sets create www.gcp.ashnova.jp. \
  --zone=YOUR_ZONE_NAME \
  --type=A \
  --ttl=300 \
  --rrdatas=$CDN_IP
```

For external DNS providers:

- Record type: `A`
- Name: `www.gcp.ashnova.jp`
- Value:
  - Production: `34.8.38.222`
  - Staging: `34.117.111.182`

```
gcloud compute ssl-certificates describe multicloud-auto-deploy-production-ssl-cert-3ee2c3ce \
  --global \
  --format="value(managed.status)"
```

If provisioning takes a long time:

- Verify that the DNS record is correctly configured
- Wait for DNS propagation (up to 48 hours, usually a few minutes to hours)
- Verify DNS resolution with `dig www.gcp.ashnova.jp`

After configuring custom domains, CORS settings for each cloud must be updated.

```
cd infrastructure/pulumi/aws
pulumi config set allowedOrigins "https://www.aws.ashnova.jp,http://localhost:5173" --stack
    production
pulumi up --stack production
```

```
cd infrastructure/pulumi/azure
pulumi config set allowedOrigins "https://www.azure.ashnova.jp,http://localhost:5173" --stack
    production
```

```
cd infrastructure/pulumi/gcp
pulumi config set allowedOrigins "https://www.gcp.ashnova.jp,http://localhost:5173" --stack
    production
pulumi up --stack production
```

```
dig www.aws.ashnova.jp
nslookup www.aws.ashnova.jp

dig www.azure.ashnova.jp
nslookup www.azure.ashnova.jp

dig www.gcp.ashnova.jp
nslookup www.gcp.ashnova.jp
```

```
curl -vI https://www.aws.ashnova.jp
curl -vI https://www.azure.ashnova.jp
curl -vI https://www.gcp.ashnova.jp

openssl s_client -connect www.aws.ashnova.jp:443 -servername www.aws.ashnova.jp < /dev/null
```

```
curl https://www.aws.ashnova.jp
curl https://www.azure.ashnova.jp
curl https://www.gcp.ashnova.jp

curl https://www.aws.ashnova.jp/health
curl https://www.azure.ashnova.jp/api/health
curl https://www.gcp.ashnova.jp/health
```

Problem: SSL certificate error occurs

Solution:

1. Check certificate status
2. Verify that domain aliases are correctly configured
3. Verify that the certificate is associated with CloudFront / Front Door / Cloud CDN

Problem: Domain does not resolve

Solution:

1. Verify that DNS records are correctly configured
2. Wait for DNS propagation (up to 48 hours)
3. Check from Google DNS: `dig @8.8.8.8 www.aws.ashnova.jp`
4. Check TTL values (after changes, wait for old TTL to expire)

Problem: Certificate remains in PROVISIONING state for a long time

Solution:

1. Verify that the DNS A record points to the correct IP address
2. Verify that the load balancer is operating normally
3. Verify that the domain is globally resolvable (run `dig` from multiple locations)

Problem: Custom domain validation fails

Solution:

1. Verify that the TXT record (`_dnsauth`) is correctly configured
2. Verify that validationToken is correct
3. Wait for DNS propagation
4. Verify with `dig TXT _dnsauth.www.azure.ashnova.jp`

Cloud	Additional Cost
AWS	ACM certificate: FreeCloudFront custom domain: Free
Azure	Front Door Managed Certificate: FreeCustom domain: Free
GCP	Managed SSL Certificate: FreeLoad balancer already exists

---

Recommended tasks after custom domain setup:

1. **Update monitoring alerts**: Configure monitoring for new domains
2. **Validate CORS settings**: Access from a browser to verify behavior
3. **Add security headers**: Configure HSTS, CSP, etc.
4. **Configure redirects**: Redirect from old endpoints to new domains
5. **Update documentation**: Add new URLs to README.md

- AWS CloudFront - Custom Domain Setup
- Azure Front Door - Custom Domain
- GCP - Managed SSL Certificates

Chapter 5

ステージング環境テスト

Last updated: 2026-02-23
Applies to: staging environments on AWS, Azure, GCP

This guide describes how to test the **landing page** and **SNS application** across all three cloud staging environments. The test suite covers:

Layer	What is tested
Landing page	CDN delivery, HTML content, HTTPS, cache headers
SNS frontend	React SPA served from CDN (`/sns/`), env-var injection
SNS API	Public endpoints (health, list posts), auth guard, full CRUD
Image upload	Presigned URL generation (S3 / Blob Storage / GCS)

Cloud	CDN URL	API URL
AWS	`https://d1tf3uumcm4bo1.cloudfront.net`	`https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com`
Azure	`https://mcad-staging-d45ihd-dseygrcbcpa3h/gjuzc1.azurewebsites.net`	`https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-`
GCP	`https://www.gcp.ashnova.jp`	`https://multicloud-auto-deploy-staging-api-son5b3ml7a-an.a.run.app`

> **Note**: The Azure API URL (`AZURE_API_URL`) is the Function App base URL **without** the `/api/HttpTrigger` path. The function uses a wildcard route `{*route}` so all API paths (e.g. `/health`, `/posts`) are served directly at the base URL.

Script	Purpose
`scripts/test-staging-all.sh`	☒ **Recommended**: orchestrates all 3 clouds
`scripts/test-landing-pages.sh`	Landing page (/) tests only

Script	Purpose
`scripts/test-e2e.sh`	Lightweight multi-cloud smoke test (v2)
`scripts/test-sns-aws.sh`	Full AWS-only test suite (authenticated)
`scripts/test-sns-azure.sh`	Full Azure-only test suite (authenticated)
`scripts/test-sns-gcp.sh`	Full GCP-only test suite (authenticated)

These checks verify connectivity and public endpoints without any login.

```
./scripts/test-staging-all.sh --quick
```

This checks:

- CDN landing page (/) returns HTTP 200
- SNS app (`/sns/`) returns HTTP 200
- API `/health` returns HTTP 200 with `{"status": "ok"}`

```
./scripts/test-landing-pages.sh
```

Tests per cloud (11 checks each):

- HTTP 200 + response time < 8s
- Content-Type: text/html
- Brand name "Ashnova" present
- Cloud badge (AWS / Azure / GCP) present
- SNS link present
- No `localhost` leak in rendered HTML
- HTTPS transport (where applicable)
- Cache-Control header
- Page size sanity (200B – 2MB)
- No server error markers
- `/sns/` path reachable

```
./scripts/test-e2e.sh
```

Public tests per cloud:

- `GET /health` → 200, `.status == "ok"`
- `GET /posts` → 200, `.items` array present
- `POST /posts` without token → 401 (auth guard)

Authenticated tests require tokens from each cloud's identity provider.

```
AWS_TOKEN=$(aws cognito-idp initiate-auth \
  --auth-flow USER_PASSWORD_AUTH \
  --client-id 1k41lqkds4oah55ns8iod30dv2 \
  --auth-parameters USERNAME=YOUR_EMAIL,PASSWORD=YOUR_PASSWORD \
  --region ap-northeast-1 \
  --query 'AuthenticationResult.AccessToken' \
  --output text)
```

```
GCP_TOKEN=$(gcloud auth print-identity-token)
```

```
AZURE_TOKEN="<paste id_token here>"
```

```
./scripts/test-staging-all.sh \
  --aws-token   "$AWS_TOKEN" \
  --azure-token "$AZURE_TOKEN" \
  --gcp-token   "$GCP_TOKEN"
```

```
./scripts/test-sns-aws.sh --token "$AWS_TOKEN"

./scripts/test-sns-azure.sh --token "$AZURE_TOKEN"

./scripts/test-sns-gcp.sh --token "$GCP_TOKEN"
```

```
./scripts/test-staging-all.sh \
  --aws-token "$AWS_TOKEN" \
  --gcp-token "$GCP_TOKEN" \
  --verbose
```

```
./scripts/test-staging-all.sh \
  --aws-token "$AWS_TOKEN" \
  --skip-cleanup
```

Section	Tests
1	CloudFront returns /, `/sns/` with HTML (Content-Type check)
2	API `/health` → 200, `.status == "ok"`
3	Auth guard: POST without token → 401
4	Authenticated: GET profile, POST/GET/PUT post
5	Presigned URLs: count=2 returned, URL starts with `https://...s3`
6	Image key validation: 16 keys → 201; 17 keys → 422
7	Cleanup: DELETE all created posts
8	React SPA env-var sanity: no `localhost` in bundle

Section	Tests
1	Front Door returns `/sns/` with React SPA HTML
2	Function App `/health` → 200, `/posts` → 200 with `.items`
3	AFD SPA deep-link URL Rewrite: `/sns/login`, `/sns/profile`, `/sns/feed` → 200
4	Auth guard: POST without token → 401
5	Authenticated: GET/POST/PUT/DELETE posts, presigned URL
6	Cleanup

Section	Tests
1	Cloud CDN returns `/sns/` with React SPA HTML (no SSR artifacts)
2	Cloud Run `/health` → 200, `/posts` → 200 with `.items`
3	Auth guard: POST/presigned-urls/profile without token → 401
4	Authenticated: GET profile, POST/GET/PUT post
5	GCS presigned URLs: count=2, URL starts with `https://storage.googleapis.com`
6	Image key validation: 16 → 201; 17 → 422
7	Cleanup

See Quick Start section above.

Lightweight smoke test with a consolidated table output. Covers 8 test points per cloud.

All scripts support URL overrides via environment variables:

Variable	Default	Used by
`AWS_CF_URL`	`https://d1tf3uumcm4bo1.cloudfront.net`	test-staging-all, test-landing-pages
`AWS_API_URL`	`https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com`	test-staging-all, test-e2e
`AZURE_FD_URL`	`https://mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net`	test-staging-all, test-landing-pages
`AZURE_API_URL`	`https://multicloud-auto-deploy-staging-func-d8a2guhfercqcqgg.japaneast-01.azurewebsites.net`	test-staging-all, test-e2e
`GCP_CDN_URL`	`https://www.gcp.ashnova.jp`	test-staging-all, test-landing-pages

Variable	Default	Used by
GCP_API_URL	https://multicloud-auto-deploy-staging-api-son5b3ml7a-an.a.run.app	test-staging-all, test-e2e

Example — test production URLs instead of staging:

```
AWS_CF_URL=https://www.aws.ashnova.jp \
AWS_API_URL=https://qkzypr32af.execute-api.ap-northeast-1.amazonaws.com \
GCP_CDN_URL=https://www.gcp.ashnova.jp \
GCP_API_URL=https://multicloud-auto-deploy-production-api-son5b3ml7a-an.a.run.app \
./scripts/test-landing-pages.sh
```

---

1. Check if the Cloud Run / Lambda / Function App is stopped:

```
# GCP
gcloud run services describe multicloud-auto-deploy-staging-api \
  --region asia-northeast1 --project ashnova \
  --format="value(status.conditions[0].status)"

# AWS
aws lambda get-function-configuration \
  --function-name multicloud-auto-deploy-staging-api \
  --region ap-northeast-1 --query State

# Azure
az functionapp show \
  --name multicloud-auto-deploy-staging-func \
  --resource-group multicloud-auto-deploy-staging-rg \
  --query state
```

2. See AI_AGENT_08_RUNBOOKS.md for redeploy procedures.

- **AWS**: Token may have expired (1h). Re-run the `aws cognito-idp initiate-auth` command.
- **GCP**: Firebase ID tokens expire in 1 hour. Re-run `gcloud auth print-identity-token`.
- **Azure**: Azure AD tokens expire in 1h. Re-extract from browser DevTools.
- Check that `AUTH_DISABLED=false` is set on the deployment (do **not** set to `true` in staging).
- **GCP**: The `GCP_SERVICE_ACCOUNT` environment variable must be set on Cloud Run. See GCP_SNS_FIX_REPORT_20260223.md (issue G5).
- **AWS**: Verify that the Lambda execution role has `s3:PutObject` on the images bucket.
- **Azure**: Verify that the Function App has the `AZURE_STORAGE_CONNECTION_STRING` env var set.
- The React SPA may not have been built/deployed to the storage bucket.
- Verify that the CI/CD pipeline ran after the last `develop` push:

```
gh run list --workflow deploy-aws.yml --limit 3
gh run list --workflow deploy-gcp.yml --limit 3
gh run list --workflow deploy-azure.yml --limit 3
```

- See AI_AGENT_06_CICD.md for deployment details.

症状: `https://staging.aws.ashnova.jp/` にアクセスすると Chrome のアドレスバーに「保護されていない通信」と表示される。

調査チェックリスト (サーバーサイドが正常かどうか確認する):

```
curl -sI https://staging.aws.ashnova.jp/ | grep -i content-security

curl -sI https://staging.aws.ashnova.jp/ | grep -i strict-transport

curl -sI http://staging.aws.ashnova.jp/ | grep -iE "http/|location"

curl -sI http://multicloud-auto-deploy-staging-frontend.s3-website-ap-northeast-1.amazonaws.com/
```

原因の切り分け:

DevTools > Security タブの表示	原因	対処
`chrome-exte...` が Secure origins に表示されている	Chrome 拡張機能が証明書エラーのあるリソースを注入している	拡張機能を1つずつオフにして特定
“You have recently allowed content with certificate errors”	過去にブラウザがこのサイトで証明書エラーを許可した記録が残っている	下記の「ブラウザキャッシュのクリア手順」参照
Mixed Content の `http://` URLが表示される	サーバーサイドに http:// リソースが残っている	JS バンドル・API レスポンス・DBを調査

ブラウザキャッシュのクリア手順 (“recently allowed certificate errors” の場合):

1. `chrome://settings/content/siteDetails?site=https%3A%2F%2Fstaging.aws.ashnova.jp` を開く
2. 「権限をリセット」と「データを消去」を実行
3. **ブラウザを完全に再起動** (タブを閉じるだけでは不十分 — メモリキャッシュが残る)
4. `https://staging.aws.ashnova.jp/` に再アクセス

> **重要**: サイトデータを消去しただけでは解消しない場合がある。Chrome はこの許可状態をメモリにキャッシュするため、ブラウザ再起動が必要。

シークレットモードで警告が消える場合: サーバーサイドは正常。ブラウザの状態 (拡張機能またはキャッシュ) が原因。

GCS objects are served via Cloud CDN with cache TTL. Force invalidation:

```
gcloud compute url-maps invalidate-cdn-cache multicloud-auto-deploy-staging-lb \
  --path "/*" --project=ashnova --async
```

See GCP_SNS_FIX_REPORT_20260223.md (issue G3).

The Front Door Rule Set rewrites `/sns/*` → `/sns/index.html`. If this fails:

1. Open Azure Portal → Front Door → Rule Sets → check the rewrite rule.
2. Verify the Rule Set is associated with the route.

The test scripts can be wired into GitHub Actions as a post-deploy verification step:

```
- name: Run staging smoke tests (public)
  run: |
    ./scripts/test-staging-all.sh --quick

- name: Run full staging tests (authenticated)
  if: secrets.AWS_TEST_TOKEN != ''
  env:
    AWS_TOKEN: ${{ secrets.AWS_TEST_TOKEN }}
    AZURE_TOKEN: ${{ secrets.AZURE_TEST_TOKEN }}
    GCP_TOKEN: ${{ secrets.GCP_TEST_TOKEN }}
  run: |
    ./scripts/test-staging-all.sh \
      --aws-token   "$AWS_TOKEN" \
      --azure-token "$AZURE_TOKEN" \
      --gcp-token   "$GCP_TOKEN"
```

Document	Description
AI_AGENT_07_STATUS.md	Current health of all 3 environments
AI_AGENT_04_API.md	API endpoint spec and request/response schema
AI_AGENT_08_RUNBOOKS.md	Step-by-step deploy / redeploy procedures
GCP_SNS_FIX_REPORT_20260223.md	GCP bug fixes (G1–G6)
AWS_SNS_FIX_REPORT_20260222.md	AWS fixes (12 bugs)
AZURE_SNS_FIX_REPORT.md	Azure fixes

Chapter 6

統合テストガイド

Created: 2026-02-18
Version: 1.0.0
Target Environments: All (AWS/GCP/Azure)

---

1. Overview
2. Test Structure
3. How to Run
4. Test Coverage
5. Troubleshooting
6. CI/CD Integration

The integration tests in this project comprehensively test the backend implementations across **3 cloud providers (AWS/GCP/Azure)**.

Test Type	Description	Tool
Unit Tests	Tests for individual backend class methods	pytest (mocked)
Integration Tests	Full-flow tests for CRUD operations	pytest (mocked)
API Endpoint Tests	HTTP tests against actually deployed APIs	pytest + requests
E2E Tests	End-to-end tests across all clouds	bash + curl

```
services/api/ ├──
 tests/ │
    ├── __init__.py                    # Test package initialization │
    ├── conftest.py                    # pytest configuration and fixtures │
```

```
    ├── test_backends_integration.py   # Backend integration tests │
    └── test_api_endpoints.py          # API endpoint tests ├──
 pytest.ini                          # pytest config file └──
 requirements-dev.txt                # Development/test dependencies

scripts/ ├──
 run-integration-tests.sh            # Python test runner script (NEW) ├──
 test-api.sh                         # Single API HTTP test ├──
 test-e2e.sh                         # Multi-cloud E2E test └──
 test-endpoints.sh                   # Endpoint health check
```

Features:

- Fixture definitions for testing
- Mock user credentials
- Sample data generation
- Cleanup processing

Key Fixtures:

```
test_user()              # Regular user
admin_user()             # Admin user
another_user()           # Different user
sample_post_body()       # Data for creating posts
sample_update_body()     # Data for updating posts
sample_profile_update()  # Data for updating profiles
aws_config()             # AWS configuration
gcp_config()             # GCP configuration
azure_config()           # Azure configuration
```

Test Classes:

Test cases common to all backends:

- ☒ `test_backend_initialization()` - Backend initialization
- ☒ `test_create_post_success()` - Create post
- ☒ `test_list_posts_empty()` - List posts (empty)
- ☒ `test_list_posts_with_tag_filter()` - Tag filtering
- ☒ `test_update_post_success()` - Update post
- ☒ `test_update_post_permission_denied()` - Permission error (update)
- ☒ `test_update_post_admin_can_update()` - Admin permission (update)
- ☒ `test_delete_post_success()` - Delete post
- ☒ `test_delete_post_permission_denied()` - Permission error (delete)
- ☒ `test_delete_post_admin_can_delete()` - Admin permission (delete)
- ☒ `test_get_profile_not_found()` - Get profile (not found)

- ☒ `test_update_profile_success()` - Update profile
- ☒ `test_get_profile_after_update()` - Get profile after update
- ☒ `test_generate_upload_urls()` - Generate upload URLs
- DynamoDB + S3 mock tests
- Marker: `@pytest.mark.aws`
- Firestore + Cloud Storage mock tests
- Marker: `@pytest.mark.gcp`
- Cosmos DB + Blob Storage mock tests
- Marker: `@pytest.mark.azure`

Test Classes:

Tests against actually deployed API endpoints:

- ☒ `test_health_check()` - Health check
- ☒ `test_list_messages_initial()` - Fetch message list
- ☒ `test_crud_operations_flow()` - Full CRUD flow
- ☒ `test_pagination()` - Pagination
- ☒ `test_invalid_message_id()` - Invalid ID (404 error)
- ☒ `test_empty_content_validation()` - Validation error

Reference: `scripts/test-api.sh` test cases 1-12

- ☒ `test_all_cloud_health_checks()` - Health check across all clouds

Reference: `scripts/test-endpoints.sh`

- ☒ `test_response_format_consistency()` - Response format consistency
- ☒ `test_api_version_consistency()` - API version consistency

Reference: consistency checks in `scripts/test-e2e.sh`

```
cd services/api
pytest tests/
```

```
pytest tests/ -m aws
```

```
pytest tests/ -m gcp
```

```
pytest tests/ -m azure
```

```
pytest tests/ -vv
```

```
pytest tests/ --cov=app --cov-report=html
```

```
pytest tests/ -k "test_create_post"
```

```
./scripts/run-integration-tests.sh
```

```
./scripts/run-integration-tests.sh -v
```

```
./scripts/run-integration-tests.sh -m aws
```

```
export AWS_API_ENDPOINT="https://abc123.execute-api.ap-northeast-1.amazonaws.com"
export GCP_API_ENDPOINT="https://app-xyz.a.run.app"
export AZURE_API_ENDPOINT="https://func-xyz.azurewebsites.net/api/HttpTrigger"

./scripts/run-integration-tests.sh --endpoints
```

```
./scripts/run-integration-tests.sh --coverage
```

```
./scripts/test-api.sh -e https://your-api-endpoint.com
```

```
./scripts/test-e2e.sh
```

```
./scripts/test-endpoints.sh
```

Method	AWS	GCP	Azure	Tests
`list_posts()`	☒	☒	☒	3
`create_post()`	☒	☒	☒	3
`update_post()`	☒	☒	☒	9
`delete_post()`	☒	☒	☒	9
`get_profile()`	☒	☒	☒	3
`update_profile()`	☒	☒	☒	6
`generate_upload_urls()`	☒	☒	☒	3

Total: 108 test cases (36 cases × 3 clouds)

Endpoint	Method	Test	Reference Script
/	GET	Health check	test-api.sh #1
/api/messages/	GET	List retrieval	test-api.sh #2, #4

Endpoint	Method	Test	Reference Script
`/api/messages/`	POST	Create	test-api.sh #3
`/api/messages/{id}`	GET	Single retrieval	test-api.sh #5
`/api/messages/{id}`	PUT	Update	test-api.sh #6, #7
`/api/messages/{id}`	DELETE	Delete	test-api.sh #8, #9
`/api/messages/?page=1`	GET	Pagination	test-api.sh #10
`/api/messages/invalid-id`	GET	Error 404	test-api.sh #11
`/api/messages/`	POST	Validation error	test-api.sh #12

Total: 27 endpoint tests (9 endpoints × 3 clouds)

Markers for classifying and filtering tests:

Marker	Description	Example
`@pytest.mark.aws`	AWS-specific tests	`pytest -m aws`
`@pytest.mark.gcp`	GCP-specific tests	`pytest -m gcp`
`@pytest.mark.azure`	Azure-specific tests	`pytest -m azure`
`@pytest.mark.integration`	Integration tests	`pytest -m integration`
`@pytest.mark.unit`	Unit tests	`pytest -m unit`
`@pytest.mark.slow`	Slow-running tests	`pytest -m "not slow"`
`@pytest.mark.requires_network`	Requires network	`pytest -m requires_network`
`@pytest.mark.requires_credentials`	Requires credentials	Excluded by default

Solution:

```
pip install pytest pytest-mock pytest-asyncio requests
```

Solution:

```
cd /workspaces/ashnova/multicloud-auto-deploy/services/api
pytest tests/
```

Solution:

```
pip install pytest-mock
```

Cause: Endpoint not configured or not yet deployed

Solution:

```
export AWS_API_ENDPOINT="https://your-endpoint.com"

pytest tests/ -m "not requires_network"
```

Cause: Test script lacks execute permission

Solution:

```
chmod +x scripts/run-integration-tests.sh
```

Example `.github/workflows/test.yml`:

```
name: Integration Tests

on:
  push:
    branches: [develop, main]
  pull_request:
    branches: [develop, main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          cd services/api
          pip install -r requirements.txt
          pip install pytest pytest-mock pytest-asyncio requests

      - name: Run integration tests
        run: |
          ./scripts/run-integration-tests.sh -v

      - name: Run endpoint tests (if deployed)
        if: env.AWS_API_ENDPOINT != ''
        env:
          AWS_API_ENDPOINT: ${{ secrets.AWS_API_ENDPOINT }}
          GCP_API_ENDPOINT: ${{ secrets.GCP_API_ENDPOINT }}
          AZURE_API_ENDPOINT: ${{ secrets.AZURE_API_ENDPOINT }}
```

```
        run: |
          ./scripts/run-integration-tests.sh --endpoints
```

```
./scripts/run-integration-tests.sh -v

./scripts/run-integration-tests.sh --coverage
```

```
cd services/api

pytest tests/ -v
```

```
./scripts/run-integration-tests.sh -m aws -v
```

```
export AWS_API_ENDPOINT="https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com"

./scripts/run-integration-tests.sh --endpoints -v
```

```
./scripts/run-integration-tests.sh --coverage
```

---

1. **Quality Assurance**: Ensure all backends operate according to specifications
2. **Regression Prevention**: Detect unintended behavior changes during code modifications
3. **Documentation**: Test code serves as usage examples for the implementation
4. **CI/CD Integration**: Automated tests perform quality checks before deployment

- ☒ **Run tests before committing**: `./scripts/run-integration-tests.sh`
- ☒ **Add tests when adding new features**: Write corresponding tests for new methods
- ☒ **Run endpoint tests after deployment**: `./scripts/run-integration-tests.sh --endpoints`
- ☒ **Run E2E tests regularly**: `./scripts/test-e2e.sh`
- ☒ **Maintain 90%+ coverage**: Check with `--coverage`

☐ Add performance tests (`TestBackendPerformance`)

☐ Add end-to-end workflow tests (`TestEndToEnd`)

☐ Load testing (Locust, etc.)

☐ Security testing (authentication & authorization)

☐ Chaos engineering tests (failure simulation)

Author: GitHub Copilot
Last Updated: 2026-02-18
Related Documents:

- API_OPERATION_VERIFICATION_REPORT.md
- AWS_BACKEND_COMPLETE_FIX_REPORT.md
- BACKEND_IMPLEMENTATION_INVESTIGATION.md

Chapter 7

Lambda Layer最適化

To reduce Lambda function deployment size, we use Lambda Layers to separate dependencies from application code.

☒ **Recommended: Using Klayers (public Lambda Layers) enables even more efficient deployments!**

See LAMBDA_LAYER_PUBLIC_RESOURCES.md for details.

- **Package size**: Over 50MB (including all dependencies)
- **Deployment method**: Upload via S3 required
- **Deployment time**: Slow due to S3 upload + Lambda update
- **Inefficient**: Every deployment re-uploads even when dependencies haven't changed
- **Package size**: A few MB (application code only)
- **Deployment method**: Direct upload possible (under 50MB)
- **Deployment time**: Completes in seconds
- **Efficient**: Dependencies managed in Layer; only update when code changes

```
Lambda Function (lightweight) ├──
 app/              # Application code (~2-5MB) │
    ├── main.py │
    ├── auth.py │
    ├── config.py │
    └── ... └──
 index.py

Lambda Layer (dependencies) └──
 python/           # Dependencies (~20-40MB)
     ├── fastapi/
     ├── pydantic/
     ├── mangum/
     ├── jwt/
```

```
    └──── ...
```

Benefits: - ☒ No build required (instantly deployable) - ☒ No maintenance needed (community managed) - ☒ Easy to update to latest version

```

cd infrastructure/pulumi/aws/simple-sns
pulumi config set use_klayers true
pulumi up
```

See LAMBDA_LAYER_PUBLIC_RESOURCES.md for details.

Benefits: - ☒ Full control (use specific versions) - ☒ Size optimization (only what's needed) - ☒ Use in private environments

```
cd /workspaces/ashnova/multicloud-auto-deploy
./scripts/build-lambda-layer.sh
```

This script performs the following: - Install only AWS-specific dependencies - Exclude boto3/botocore (included in Lambda runtime) - Exclude Azure/GCP SDKs (not needed for AWS) - Remove test files and documentation to reduce size - Generate `services/api/lambda-layer.zip`

```
cd infrastructure/pulumi/aws/simple-sns

pulumi config set use_klayers false

pulumi up
```

Pulumi will automatically: - Create the custom Lambda Layer - Deploy only application code to the Lambda function - Attach the Layer to the Lambda function

```
gh workflow run deploy-aws.yml
```

The GitHub Actions workflow automatically:

1. **Fetch ARN**: Uses the latest Klayers ARN
2. **Package application code**: ZIPs code only
3. **Update Lambda function**:
   - Package under 50MB: Direct upload ☒
   - Package over 50MB: Via S3 (fallback)
4. **Attach Klayers**: Connect the public Layer to Lambda

```
name: Deploy
on:
  workflow_dispatch:
    inputs:
      use_klayers:
        description: "Use Klayers (public Lambda Layers)"
```

```
        default: true  # Use Klayers by default
```

Select `use_klayers: false` in the GitHub Actions workflow:

1. **Build Layer**: Run `build-lambda-layer.sh`
2. **Package application code**: ZIPs code only
3. **Deploy custom Layer**: Publish as Lambda Layer
4. **Update Lambda function**: Direct upload or via S3
5. **Attach Layer**: Connect custom Layer to Lambda

```
fastapi==0.115.0
pydantic==2.9.0
pydantic-settings==2.5.2
python-jose[cryptography]==3.3.0
python-multipart==0.0.9
pyjwt==2.9.0
mangum==0.17.0
requests==2.32.3
```

- **boto3/botocore**: Included in Lambda runtime
- **Azure SDK**: Not required for AWS
- **GCP SDK**: Not required for AWS
- **PostgreSQL/SQLAlchemy**: DynamoDB only is used

```
Lambda package: 65MB (code + all dependencies) └──
 S3 deployment required
```

```
Lambda package: 3MB (code only) ├──
 Direct upload possible └──
 Layer: 25MB (dependencies)
     └── Re-deploy only when dependencies change
```

```
./scripts/build-lambda-layer.sh

ls -lh services/api/lambda-layer.zip
```

```
aws lambda get-function-configuration \
  --function-name multicloud-auto-deploy-staging-api \
  --query 'Layers[*].Arn'
```

Lambda Layer limits: - **ZIP size**: 50MB - **Unzipped size**: 250MB

Size reduction methods: 1. Remove unnecessary files in `build-lambda-layer.sh` 2. Use `--no-deps` to exclude extra dependencies 3. Remove test files and documentation

- Pin dependency versions
- Only update Lambda when application code changes
- Only update Layer when dependencies change

```
aws lambda list-layer-versions \
  --layer-name multicloud-auto-deploy-staging-dependencies

aws lambda delete-layer-version \
  --layer-name multicloud-auto-deploy-staging-dependencies \
  --version-number 1
```

```
LAYER_NAME: multicloud-auto-deploy-staging-dependencies

LAYER_NAME: multicloud-auto-deploy-production-dependencies
```

- Lambda Layer Public Resources Guide ☒ **Recommended**
- Klayers GitHub
- Klayers API
- AWS Lambda Layers Documentation
- Lambda Deployment Packages
- Lambda Quotas

☒ **We strongly recommend using Klayers (public Layers)**

By using Lambda Layers:

☒ Reduces deployment size from **65MB → 3MB**
☒ Reduces deployment time from **minutes → seconds**
☒ S3 upload **no longer required**
☒ Dependency management is **separated** for efficiency
☒ Build time is **zero** (when using Klayers)

Selection Criteria:

Scenario	Recommended Approach
Standard development / production	**Klayers** ☒
Rapid prototyping	**Klayers** ☒
Specific version required	Custom Layer
Private dependencies	Custom Layer
Minimize size to the extreme	Custom Layer

See Lambda Layer Public Resources Guide for details.

Chapter 8

アーキテクチャマッパー

This tool collects current resource information from AWS, Azure, and GCP, normalizes it into JSON, and renders a Mermaid system architecture diagram.

- Build a single architecture snapshot from live cloud resources
- Cross-check Pulumi expected state vs actual cloud resources
- Generate a portable diagram artifact (`.mmd`) for documentation and reviews
- `scripts/cloud_architecture_mapper.py`
- `scripts/refresh-architecture.sh`
- Pulumi CLI authenticated (`pulumi login`)
- AWS CLI authenticated (`aws sts get-caller-identity`)
- Azure CLI authenticated (`az account show`)
- GCP CLI authenticated (`gcloud auth list`)
- Access to stack outputs for:
  - `infrastructure/pulumi/aws`
  - `infrastructure/pulumi/azure`
  - `infrastructure/pulumi/gcp`

```
python3 scripts/cloud_architecture_mapper.py collect \
  --environment staging \
  --output docs/generated/architecture/snapshot.json
```

```
python3 scripts/cloud_architecture_mapper.py render \
  --input docs/generated/architecture/snapshot.json \
  --output docs/generated/architecture/architecture.mmd

python3 scripts/cloud_architecture_mapper.py render \
  --input docs/generated/architecture/snapshot.json \
  --output docs/generated/architecture/architecture.mmd \
  --validate
```

```
python3 scripts/cloud_architecture_mapper.py all \
  --environment staging \
  --snapshot-output docs/generated/architecture/snapshot.json \
  --diagram-output docs/generated/architecture/architecture.mmd \
  --validate
```

```
./scripts/refresh-architecture.sh staging
```

```
python3 scripts/cloud_architecture_mapper.py collect \
  --environment staging \
  --output docs/generated/architecture/snapshot.staging.json

python3 scripts/cloud_architecture_mapper.py collect \
  --environment production \
  --output docs/generated/architecture/snapshot.production.json

python3 scripts/cloud_architecture_mapper.py compare \
  --staging-input docs/generated/architecture/snapshot.staging.json \
  --production-input docs/generated/architecture/snapshot.production.json \
  --output docs/generated/architecture/architecture-combined.mmd \
  --validate
```

- `architecture: refresh (staging)`
- `architecture: validate (render)`
- `docs/generated/architecture/snapshot.json`
  - `clouds.<provider>.pulumiOutputs`: Pulumi stack outputs
  - `clouds.<provider>.resources`: normalized resources
  - `clouds.<provider>.errors`: collection warnings/errors
- `docs/generated/architecture/architecture.mmd`
  - Mermaid `flowchart LR` diagram with provider subgraphs
- AWS: CloudFront, S3, API Gateway v2, Lambda, DynamoDB
- Azure: Front Door profile, Storage Account, Function App, Cosmos DB
- GCP: Backend Bucket / URL Map, Cloud Run, GCS, Firestore
- The collector is intentionally read-only and uses list/get APIs.
- If one cloud collection fails, the tool still writes snapshot/diagram with warnings.
- `--validate` checks Mermaid node ID safety and edge references, and exits with non-zero status when invalid.
- This is an MVP foundation for future diff and drift analysis.

`scripts/generate_icon_diagram.py` generates interactive HTML architecture diagrams with official cloud service icons embedded. This tool enhances the basic Mermaid diagrams with:

- **Official SVG icons** from AWS, Azure, and GCP
- **Interactive flowchart nodes** with icons embedded in-node (top-left corner)
- **Base64-encoded data URIs** for zero external dependencies
- **Responsive sidebar legend** with resource counts
- ☒ Official AWS Architecture Icons (48px SVG, Q3-2021 package)
- ☒ Official Azure Public Service Icons V14 (18px SVG with gradients)
- ☒ Official GCP Product Icons (SVG)
- ☒ JavaScript icon injection into Mermaid-rendered nodes
- ☒ **Dual icon placement**: Top-left corner (24px) + Text-inline (20px)
- ☒ No external CDN dependencies (all icons embedded)
- ☒ Separate diagrams for staging, production, and combined view

Icons are stored in `assets/icons/{provider}/`:

```
assets/icons/ ├──
 aws/ │
    ├── cloudfront.svg      (AWS CloudFront) │
    ├── lambda.svg          (AWS Lambda) │
    ├── s3.svg              (Amazon S3) │
    ├── dynamodb.svg        (Amazon DynamoDB) │
    └── api-gateway.svg     (Amazon API Gateway) ├──
 azure/ │
    ├── cdn.svg             (Azure Front Door) │
    ├── function.svg        (Azure Functions) │
    ├── storage.svg         (Azure Storage Account) │
    └── cosmos-db.svg       (Azure Cosmos DB) └──
 gcp/
     ├── cdn.svg             (Cloud CDN)
     ├── run.svg             (Cloud Run)
     ├── storage.svg         (Cloud Storage)
     ├── firestore.svg       (Cloud Firestore)
     └── load-balancer.svg   (Cloud Load Balancer)
```

Total size: ~50KB (14KB AWS + 16KB Azure + 20KB GCP) **Format**: SVG (embedded as Base64 data URIs in HTML) **Source**:

- AWS: AWS Architecture Icons Asset Package
- Azure: Azure Public Service Icons V14
- GCP: Google Cloud Icons

```
python3 scripts/generate_icon_diagram.py staging
```

```

python3 scripts/generate_icon_diagram.py production

python3 scripts/generate_icon_diagram.py combined
```

Generated HTML files in `docs/generated/architecture/`:

- `architecture.staging.html` (~85KB) - Staging environment with official icons
- `architecture.production.html` (~85KB) - Production environment with official icons
- `architecture-combined.html` (~91KB) - Staging + Production overlay with color-coded nodes

Each HTML file includes:

- Embedded Mermaid.js v10 (ESM module)
- Base64-encoded SVG icons (no external requests)
- JavaScript DOM manipulation for icon injection
- Interactive legend sidebar with resource counts
- Responsive layout (CSS Grid)

Icon Loading:

```python
def load_official_icon(provider: str, resource_type: str) -> str:
    """Load official SVG icon and return Base64 data URI"""
    # Icon mappings for AWS, Azure, GCP
    # Returns: ...
```

Icon Injection (JavaScript):

```javascript
// Pattern: flowchart-{provider}_{resource_type}-{name}-{index}
const nodeId = node.id;
const match = nodeId.match(/flowchart-([a-z]+)_([a-z_]+)-/);
if (match && resourceIcons[`${match[1]}:${match[2]}`]) {
  // 1. Top-left corner icon (24x24px at x+6, y+6)
  const topIcon = document.createElementNS(
    "http://www.w3.org/2000/svg",
    "image",
  );
  topIcon.setAttribute("x", rectX + 6);
  topIcon.setAttribute("y", rectY + 6);
  topIcon.setAttribute("width", 24);
  topIcon.setAttribute("height", 24);
  node.appendChild(topIcon);

  // 2. Text-inline icon (20x20px, 4px left of text)
  const textIcon = document.createElementNS(
    "http://www.w3.org/2000/svg",
    "image",
  );
  textIcon.setAttribute("x", textX - 24); // 4px left of text
```

```
  textIcon.setAttribute("y", textY);
  textIcon.setAttribute("width", 20);
  textIcon.setAttribute("height", 20);
  labelGroup.insertBefore(textIcon, labelGroup.firstChild);
}
```

Dual Icon Placement:

- **Top-left corner**: 24x24px icon at (rect.x + 6, rect.y + 6) for quick resource type identification
- **Text-inline**: 20x20px icon placed 4px left of node label text for enhanced readability
- Both icons use the same provider-specific official SVG

Node ID Convention:

- Pattern: `flowchart-{provider}_{resource_type}-{name}-{index}`
- Examples: `flowchart-aws_cdn-E1TBH4R432SZBZ-0`, `flowchart-azure_compute-mcadfuncdiev0w-1`
- Provider: `aws` | `azure` | `gcp`
- Resource types: `cdn`, `object_storage`, `serverless_function`, `database`, `api_gateway`, `load_balancer`, `compute`
- Icons assume 48x48px source size (scaled to 24px top-left, 20px text-inline)
- Injection timing requires 800ms setTimeout for Mermaid rendering
- No icon caching (data URIs embedded in each HTML file)
- Resource type mapping is manually maintained in `load_official_icon()`
- Text-inline icon positioning may vary depending on Mermaid node type (foreignObject vs text elements)

The tool can identify isolated/orphaned resources (nodes with no connections). As of 2026-02-27:

13 isolated resources identified across staging:

- ☒ **Delete candidates** (5): `multicloud-auto-deploy-staging-posts` (AWS DynamoDB), 4 Azure Storage Accounts not in Pulumi
- ☒ **In use but unconnected** (2): GCP function-source bucket, uploads bucket
- ☒ **Needs verification** (6): 3 AWS CloudFront distributions for ashnova.jp domains, GCP landing bucket, 2 others

> **Note**: Resource cleanup postponed pending further investigation (2026-02-27)